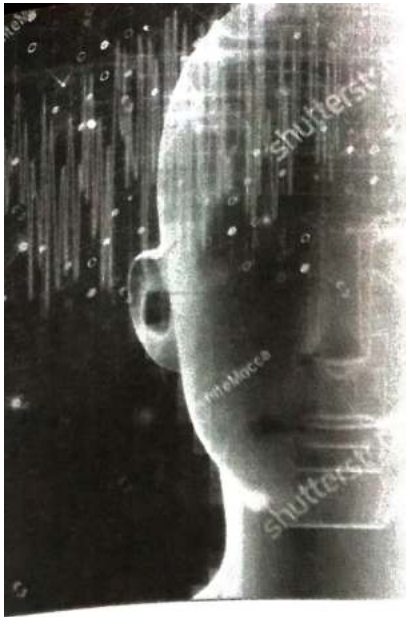




Chapter 11

Support Vector Machines

"Science is the systematic classification of experience."
— George Henry Lewes, *Physical Basis of Mind*

Support vector machines (SVM) are extremely popular classifiers. SVM is the default choice classifier for most practical applications. It can be extended to solve regression problems as well.

Learning Objectives

- Introduce the concept of support vector machines and its advantages
- Derive hyperplanes and margins
- Introduce separable support vector machines
- Outline Lagrangian primal and dual optimization problems
- Explain soft margin support vector machines
- Illustrate the use of kernels and its types
- Introduce the concept of support vector Kernel Regression

11.1 INTRODUCTION TO SUPPORT VECTOR MACHINES

Scan for information on 'Decision Functions' and for 'Additional Examples'



SVM is a supervised learning algorithm that takes a labelled data as input and creates learning functions that can be used for classification of unknown test data as shown in Figure 11.1.

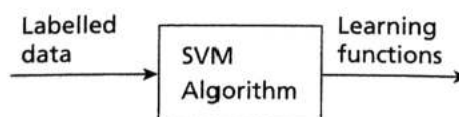


Figure 11.1: SVM Algorithm

Vladimir Vapnik, one of the main developers of the **Vapnik–Chervonenkis** theory of statistical learning, and the co-inventor of the support-vector machine method and support-vector clustering algorithm, provided a sound theoretical foundation for SVM. SVM is an extremely popular classifier and used in many practical applications. There are many advantages of SVM:

1. SVM can perform as a linear and non-linear classifier.
2. SVM can perform regression.
3. The decision boundary can be constructed using a small set of support vectors and can be constructed using less training samples. But this factor is dependent on the nature of the given application.
4. SVM works with higher dimensional data.
5. Generally, SVM classifiers are very robust and immune to the data features or dimensions.
6. SVM can be implemented in many applications such as object recognition, face recognition, Iris classification, and Pedestrian recognition successfully.

The aim of SVM is to produce a decision plane that defines the boundary between classes to classify the data points. In contrast to the decision theoretic minimum distance classifier that uses only one decision boundary, SVM uses a reference hyperplane and two decision boundaries to classify the points. This is the major difference between SVM and all other classifiers. This is shown in Figure 11.2 where there is a reference hyperplane shown with a dark line and two parallel boundaries in dotted lines.

The decision boundaries are drawn with the following characteristics:

- The decision boundary should be as far away from the data points, and hence the distance should be maximized between the line and the nearest data point.
- Should avoid misclassifications of data.

The first constraint indicates that the hyperplane should provide a wide margin. SVM is different from other classifiers as the focus of SVM is to maximize the margin as much as possible. A margin is defined as the amount of space between two classes as defined by the hyperplane. Formally, the distance between the hyperplanes is known as the margin of the classifier. The margin should be as large as possible. The focus of SVM is to obtain maximal margin classifier (MMH) often called 'hard SVM'.

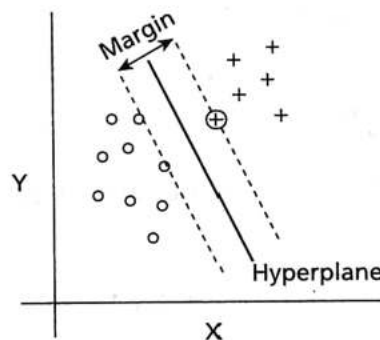


Figure 11.2: Margin Shown with Supporting Boundary Lines

It can be observed from Figure 11.1 that the selection of hyperplane is dependent on 'Support vectors'. Support vectors are those data points that fall on the supporting boundary lines. In a training dataset, all training samples that fall on the boundary lines are called support vectors.

A SVM implements a binary classifier. This means that there are only two classes, say +1 and -1. Later on, the idea of extending the two class SVM to multiclass SVM is discussed in the chapter. As of now, let us consider only two class problems.

Let us consider a dataset:

$$D = \{(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)\}, x \in \mathbb{R}^n, \text{ and } y \text{ is the target label. } y = \{-1, +1\}$$

The aim of a linear maximal margin support vector machine or Hard-margin SVM is to find a hyperplane that maximally separates the classes. From Box 11.1, one can check out that a hyperplane is an n -dimensional generalization of a line which is a set of points that satisfy the hyperplane equation:

$$h(x) = b + w_1x_1 + w_2x_2 + \dots + w_nx_n = 0 \\ \text{or } b + w^T x = 0 \quad (11.1)$$

where, b is the intercept and $a_1, a_2, \dots, a_n$ are coefficients and n is the dimension of the data point.

For a simple 2D, the hyperplane can be written as:

$$w \cdot x + b = 0 \quad (11.2)$$

Then, the hyperplane equation separates the data points x_i into two classes +1 and -1. This hyperplane equation can also be written as: $w^T x = 0$ (See Box 11.1). Here, w is the weight vector and b is the bias or offset from the origin. This equation can be expressed better in terms of the dot vector product as $w \cdot x = 0$ for dimensions three and above.

The difference between SVMs and all other classifiers such as a decision theoretic classifier is that the SVM uses two decision lines constructed using the reference hyperplane. Two lines act as two classifiers. This is written as follows:

$$H_1: w \cdot x_i + b \geq 0 \text{ for } y = +1 \text{ and} \\ H_2: w \cdot x_i + b < 0 \text{ for } y = -1 \quad (11.3)$$

This classifier is suitable for two class problems. One can also bring the output y into the Eq. (11.3) and that changes the equation to:

$$h(x) = y_i(w \cdot x_i + b) \geq 1 \text{ for } i = 1, 2, \dots, n \text{ or one can alternatively can find predictions as:} \\ h(x_i) = \text{sign}(w \cdot x_i + b) \quad (11.4)$$

The sign of $h(x)$ is always positive if it is correctly classified. Its value is negative if it is wrongly classified. The basics of hyperplane are given in Box 11.1.

11.2 OPTIMAL HYPERPLANE

What is an optimal hyperplane? Consider the following Figure 11.3, where a decision function separates the samples.

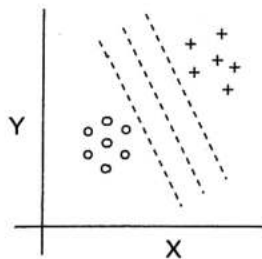


Figure 11.3: Multiple Hyperplanes for Separating the Data

There can be many hyperplanes separating the samples. Recollect that a hyperplane divides the input space into two half spaces, so that all samples (°) fall below the hyperplane and samples (+) fall above the line. The aim is to find the maximum margin. Each boundary is associated with a pair of hyperplanes, say h_1 and h_2 . These parallel hyperplanes are obtained by moving the hyperplane h_1 till it touches (+), and h_2 till it touches (°). The hyperplane that creates the maximum margin is called optimal hyperplane.

The basics required for constructing the hyperplane are referred in Box 11.1.

Box 11.1: Basics of Constructing a Hyperplane

Any point $v = (v_1, v_2)$, $v \neq 0$, can be defined as a vector in a plane starting at the origin and ending at the given point. For example, the point (4, 3) can be represented as a vector shown in Figure 11.4.

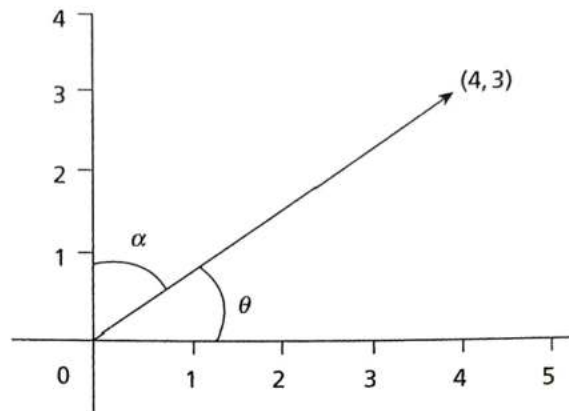


Figure 11.4: A Point (4, 3) in Vector Form

The vector, as discussed earlier, is an object that has both length and direction. The calculation of size of length of the vector is often required in machine learning. The length of the vector is called vector norm or vector magnitude. The length of the vector is often a positive number. The form of the vector norm is given as $\|v\|$. The L_2 norm or Euclidean norm is calculated as the Euclidean distance from the origin. It is expressed as the square root of the sum of squared vector values:

$$\|v\|_2 = \sqrt{a_1^2 + a_2^2} \quad (11.5)$$

Hyperplane Equations

All are familiar with line equation:

$$y = ax + b \quad (11.6)$$

Rewriting x and y as x_1 and x_2 in Eq. (11.6) gives the equation,

$$x_2 = a \times x_1 + b, \quad a \times x_1 - x_2 = 0. \quad (11.7)$$

This Eq. (11.7) can be written as $w^T x = 0$. Here, the vector w determines the orientation of the hyperplane and b is the bias or intercept or offset from the origin. If the weight vectors $w = (b, a, -1)^T$ and $x = (1, x_1, x_2)^T$, then one can check $w^T x = y - ax - b = 0$. Thus, these equations are equivalent.

(Continued)

One can use vector dot notation $w \cdot x = 0$ instead of $w^T x = 0$ as the above vectors w and x are equivalent to $w \cdot x = 0$. This is suitable for data whose dimension is 3 or more.

The distance of a point x and the hyperplane, say $h(x)$, is given as $r = \frac{h(x)}{\|w\|}$. Here, r is called directed distance. The aim of SVM is to maximize r for the points that are closest to the optimal hyperplane. These points are called support vectors. The intuition is that the best or optimal hyperplane is that falls exactly in the middle of the two classes of data. ' r ' is positive if x is on the positive side of the hyperplane and it is negative if it is on the negative side of the hyperplane. One can multiply it by class labels as $[+1, -1]$ for binary classifiers. This gives:

$$yr = \frac{yh(x)}{\|w\|} \quad (11.8)$$

The expression can be simplified as the numerator just indicates the class $+1$ or -1 and the denominator does not change at all. So, one can take the absolute value to remove the numerator of Eq. (11.8) as 1. So, the simplified expression is:

$$|r| = \frac{1}{\|w\|} \quad (11.9)$$

The aim is, therefore, to maximize r of the points that are closest to the hyperplane. Such data points are called support vectors.

11.3 FUNCTIONAL AND GEOMETRIC MARGIN

Given the dataset D , one can compute the functional margin f . Functional margin is a function that can indicate the classification or misclassification of a data point, but it cannot indicate whether the point is close or far from the hyperplane. The functional margin of a data or example is computed as follows:

$$f = \min_{i=1 \dots n} f_i$$

Here, f is called the functional margin of example or data.

And functional margin of the entire dataset F can be computed as follows:

$$F = \min_{i=1 \dots n} y(w \cdot x_i + b) \quad (11.10)$$

The closeness or farness of the data point from the hyperplane is indicated through the geometric margin by the magnitude of the distance in terms of $\|w\|$. In short, the geometric margin is a scaled version of the functional margin by a factor of $\|w\|$. The normalized equation of $f = y(w \cdot x) + b$ becomes:

$$\gamma = y \left(\frac{w}{\|w\|} \cdot x + \frac{b}{\|w\|} \right) \quad (11.11)$$

For the dataset, one can say, $M = \min_{i=1 \dots n} \gamma_i$. Here, γ is called the geometric margin of dataset D . The relationship between the functional and geometric margin is given as:

$$M = \frac{F}{\|w\|} \quad (11.12)$$

Here, M is called geometric margin of the entire dataset.

Maximizing the geometric margin (M) does not depend on the scale of w and b (Geometric margin is scale invariant) and hence one can rescale w and b such that $F = 1$. Therefore, the geometric margin of Eq. (11.12) can be given as:

$$M = \frac{F}{\|w\|} = \frac{1}{\|w\|} \quad (11.13)$$

The distance between any point and hyperplane is $\frac{1}{\|w\|}$, where, $\|w\|$ is the norm. This is equal to the distance from any point on the other side as also $\frac{1}{\|w\|}$. Therefore, the maximum margin is given as $\frac{2}{\|w\|}$. The maximal margin classifier now can be expressed as an optimization problem as:

$$\text{Max } \frac{2}{\|w\|} \text{ subjected to the constraint } y_i (w \cdot x_i + b) \geq 1, \forall x_i \in D. \quad (11.14)$$

Maximizing $\frac{2}{\|w\|}$ is equivalent to minimizing $\|w\|$. If $\|w\|$ is large, then $\frac{2}{\|w\|}$ is small and when $\|w\|$ is small, then $\frac{2}{\|w\|}$ is large. Since our aim is to maximize $\frac{2}{\|w\|}$, $\|w\|$ serves as an error function called distance error function.

If the norm of the line is large, then $\frac{2}{\|w\|}$ is small and vice versa. Since large distance is preferable, one must find a line whose norm is small. If so, the constructed classifier is good. The following problem will help to understand the role of the distance error function.

Example 11.1: The hyperplane function for two variables is $b + a_1x_1 + a_2x_2$. If two hyperplanes given are for classifier 1 as $5 + 2x_1 + 5x_2$ and $5 + 20x_1 + 50x_2$, for classifier 2. Find the distance error function and pick a good classifier constructed using these hyperplanes?

Solution: This norm is the distance error. It is given as:

$$\sqrt{a_1^2 + a_2^2} \quad (11.15)$$

The weight vector for the first equation omitting the intercept is $(2, 5)$, and using Eq. (11.15), one get the norm as:

$$\|w\| = \sqrt{2^2 + 5^2}, \text{ which is approximately } 5.39.$$

For the second equation with weight vector $(20, 50)$, using Eq. (11.15),

$$\|w\| = \sqrt{20^2 + 50^2} = 53.85$$

The distance between the lines can be calculated for classifier 1, as $\frac{2}{\|w\|} = \frac{2}{5.39} = 0.37$, and for the second one as $\frac{2}{53.85} = 0.037$. It can be noticed that if the distance error is large, then $\frac{2}{\|w\|}$ is small and vice versa. Since distance error 5.39 is smaller than 53.85, the first equation is preferable and hence the classifier that uses this hyperplane is a good classifier.

Example 11.2: Points (4, 1), (4, -1) and (6, 0) belong to class positive and points (1, 0), (0, 1) and (0, -1) belong to negative class. Draw an optimal hyperplane to classify the points.

Solution: The scatter plot of the data points is shown in Figure 11.5.

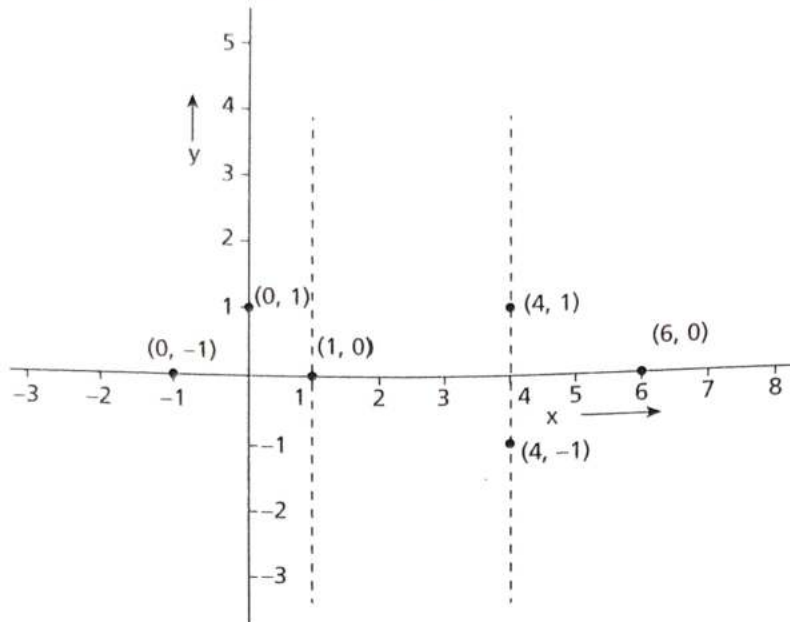


Figure 11.5: Scatter Plot of the Points with the Support Vectors (1, 0), (4, 1) and (4, -1)

It can be observed that the support vectors are (1, 0), (4, 1) and (4, -1) as shown below:

$$s_1 = \begin{pmatrix} 1 \\ 0 \end{pmatrix}, s_2 = \begin{pmatrix} 4 \\ 1 \end{pmatrix}, s_3 = \begin{pmatrix} 4 \\ -1 \end{pmatrix}$$

The augmented vector can be obtained by adding the bias given as follows:

$$\tilde{s}_1 = \begin{pmatrix} 1 \\ 0 \\ 1 \end{pmatrix}, \tilde{s}_2 = \begin{pmatrix} 4 \\ 1 \\ 1 \end{pmatrix}, \tilde{s}_3 = \begin{pmatrix} 4 \\ -1 \\ 1 \end{pmatrix}$$

From these, a set of three equations can be obtained based on these three support vectors as follows:

$$\begin{aligned} \alpha_1 \tilde{s}_1 \tilde{s}_1 + \alpha_2 \tilde{s}_2 \tilde{s}_1 + \alpha_3 \tilde{s}_3 \tilde{s}_1 &= -1 \\ \alpha_1 \tilde{s}_1 \tilde{s}_2 + \alpha_2 \tilde{s}_2 \tilde{s}_2 + \alpha_3 \tilde{s}_3 \tilde{s}_2 &= +1 \\ \alpha_1 \tilde{s}_1 \tilde{s}_3 + \alpha_2 \tilde{s}_2 \tilde{s}_3 + \alpha_3 \tilde{s}_3 \tilde{s}_3 &= +1 \end{aligned} \quad (11.16)$$

It can be observed that in equation 1, $\tilde{s}_1$ is constant, in equation 2, $\tilde{s}_2$ is constant and in equation 3, $\tilde{s}_3$ is constant. The first equation is for (1, 0) that belongs to the negative class -1 and equation 2 and equation 3 are for the positive class +1.

Substituting the augmented support vectors in the Eq. (11.16) yields:

$$\begin{aligned} \alpha_1 \begin{pmatrix} 1 \\ 0 \\ 1 \end{pmatrix} \begin{pmatrix} 1 \\ 0 \\ 1 \end{pmatrix} + \alpha_2 \begin{pmatrix} 4 \\ 1 \\ 1 \end{pmatrix} \begin{pmatrix} 1 \\ 0 \\ 1 \end{pmatrix} + \alpha_3 \begin{pmatrix} 4 \\ -1 \\ 1 \end{pmatrix} \begin{pmatrix} 1 \\ 0 \\ 1 \end{pmatrix} \\ = 2\alpha_1 + 5\alpha_2 + 5\alpha_3 = -1 \end{aligned}$$

$$\begin{aligned}
& \alpha_1 \begin{pmatrix} 1 \\ 0 \\ 1 \end{pmatrix} \begin{pmatrix} 4 \\ 1 \\ 1 \end{pmatrix} + \alpha_2 \begin{pmatrix} 4 \\ 1 \\ 1 \end{pmatrix} \begin{pmatrix} 4 \\ 1 \\ 1 \end{pmatrix} + \alpha_3 \begin{pmatrix} 4 \\ -1 \\ 1 \end{pmatrix} \begin{pmatrix} 4 \\ 1 \\ 1 \end{pmatrix} \\
&= 5\alpha_1 + 18\alpha_2 + 16\alpha_3 = +1 \\
& \alpha_1 \begin{pmatrix} 1 \\ 0 \\ 1 \end{pmatrix} \begin{pmatrix} 4 \\ -1 \\ 1 \end{pmatrix} + \alpha_2 \begin{pmatrix} 4 \\ 1 \\ 1 \end{pmatrix} \begin{pmatrix} 4 \\ -1 \\ 1 \end{pmatrix} + \alpha_3 \begin{pmatrix} 4 \\ -1 \\ 1 \end{pmatrix} \begin{pmatrix} 4 \\ -1 \\ 1 \end{pmatrix} \\
&= 5\alpha_1 + 16\alpha_2 + 18\alpha_3 = +1
\end{aligned}$$

Solving these three simultaneous equations with three unknowns yields the values:

$$\alpha_1 = -3$$

$$\alpha_2 = +1$$

$$\alpha_3 = 0$$

The optimal hyperplane vectors are given as:

$$\begin{aligned}
w &= \sum_{i=1}^3 \alpha_i \times \tilde{s}_i \\
&= -3 \times \begin{pmatrix} 1 \\ 0 \\ 1 \end{pmatrix} + 1 \times \begin{pmatrix} 4 \\ 1 \\ 1 \end{pmatrix} + 0 \times \begin{pmatrix} 4 \\ -1 \\ 1 \end{pmatrix} = \begin{pmatrix} 1 \\ 1 \\ -2 \end{pmatrix} \quad (11.17)
\end{aligned}$$

The hyperplane is $(1, 1)$ with an offset -2 . The optimal hyperplane is shown in Figure 11.6.

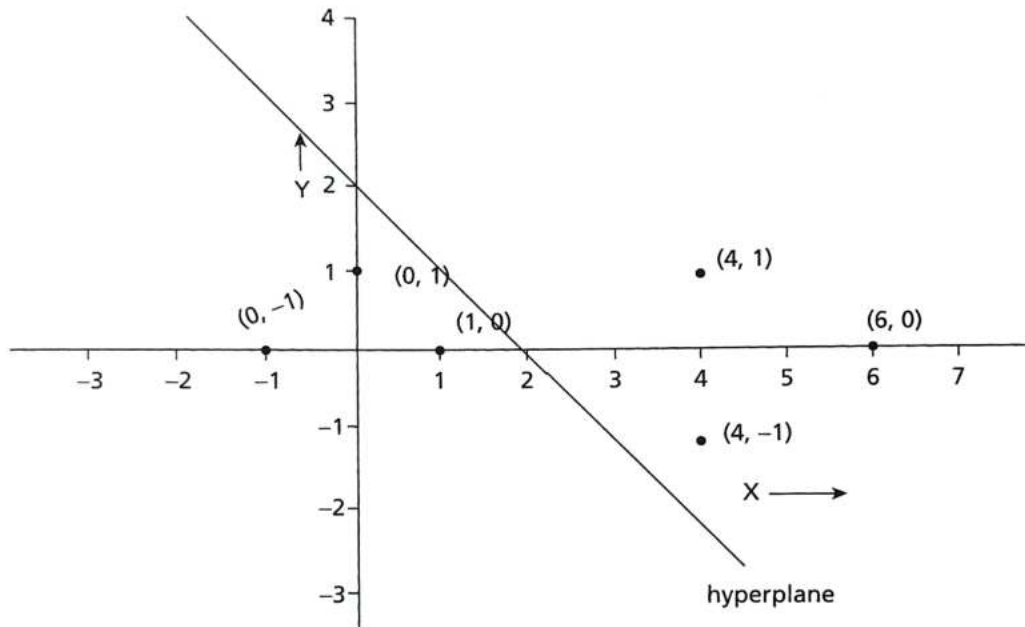


Figure 11.6: Scatter Plot of the Points with Hyperplane

11.4 HARD MARGIN SVM AS AN OPTIMIZATION PROBLEM

In example 11.2, the data points are less. So, one can visually find out the support vectors easily and construct the hyperplane. In practice, it is not feasible as involved data is huge. So, the method

of constructing SVM is to reformulate it as an optimization problem. The method of constructing a solution using optimal problem is as follows:

1. State the problem as a primal optimization problem.
2. Convert that to dual Lagrangian problem. Finally, using the solutions, construct the hyperplanes.

Any problem whose objective function needs to be maximized or minimized, subject to the constraints, is called an optimization problem. Since SVM involves maximization of margin, it can be modelled as an optimization problem.

Maximizing $\frac{2}{\|w\|}$ is equivalent to minimizing $\|w\|$ as minimization problems are relative to solve than maximization problems. So, the trick is to get a minimization of $\frac{1}{2}\|w\|^2$ to cancel the factor 2 in the numerator. The resultant is a minimization problem of the following objective function and can be formulated as follows:

$$\text{Minimize}_{w, b} \frac{1}{2}\|w\|^2 \quad (11.18)$$

An objective function that has no constraint is called an unconstrained optimization problem (See the Box 11.2). So, more constraints are required to convert that into an optimization problem of proper form. Two constraints are suitable for this problem and based on these the hyperplane can be constructed as follows:

$$H_1: w \cdot x_i + b \geq 1, \forall y_i = 1$$

$$H_2: w \cdot x_i + b < 1, \forall y_i = -1$$

Here, y_i is the ground truth, and w and x are weight and input vectors, respectively, combined by dot product. These two constraints can be combined into one constraint as $y_i(w \cdot x_i + b) - 1 \geq 0$ for $\forall x_i \in D$. The dot product $w \cdot x_i$ can also be written as $w^T \cdot x_i$. The dot product notation is used in this book for consistency. The construction of parallel lines is not difficult as the parallel lines have same weights but different bias.

The optimization problem now can be stated by adding the constraint to Eq. (11.18) as follows:

$$\text{Minimize}_{w, b} \frac{1}{2}\|w\|^2$$

Subject to the constraint:

$$y_i(w \cdot x_i) + b - 1 \geq 0, \text{ for } \forall x_i \in D$$

This is called *primal optimization problem*.

Since the objective function is quadratic and the constraints are linear, this problem is called as convex optimization problem. Lagrange multiplier method is a popular method for solving convex optimization problems.

11.4.1 Lagrangian Optimization Problem

Lagrange multipliers are one way of solving the optimization problem with respect to equality constraints. The formulation for the Lagrangian can be given as follows:

Lagrangian problem can be formulated as:

$$\mathcal{L} = \mathcal{L}(w, b, \alpha) = \text{Min}_{w, b} \frac{1}{2} \|w\|^2 - \sum_{i=1}^n \alpha_i [y_i (w \cdot x_i + b) - 1] \quad (11.19)$$

Subject to $\alpha_i \geq 0$.

Here, α_i is called a Lagrangian multiplier. To solve the Lagrangian multipliers, one must take the partial derivatives of $\mathcal{L}$ with respect to weights w and bias b and equate it to zero.

The Lagrangian problem can now be formulated as a problem as follows:

$$\begin{aligned} &\text{Minimize}_{w, b} \max_{\alpha} \mathcal{L}(w, b, \alpha) \\ &\text{subject to } \alpha_i \geq 0 \text{ For } \forall_i, i = 1 \dots n. \end{aligned} \quad (11.20)$$

It can be observed that the objective function of Lagrangian equation is modified. The reason is that the function is minimized when $w = 0$. Such a solution violates the constraints. That is why the objective function is modified to incorporate the constraint in the original equation. The Lagrangian multipliers are support vectors. To minimize the Lagrangian, one must take the derivative of $\mathcal{L}$ with respect to w and b and set them to zero x_i , support vector, so then its corresponding Lagrangian vector is greater than zero. If Lagrangian vectors are zero, then it means that these are not support vectors.

The Lagrangian function works only if the constraint is linear. Since the constraints involved in SVM are inequality constraints, the Lagrangian method cannot be applied directly. Hence, the focus is on another constraint called KKT constraints. KKT or Karush-Kuhn-Tucker (KKT) conditions modify the inequality constraints into a set of equality constraints. Such a transformation leads to constraints that satisfy the KKT conditions.

Box 11.2: Fundamentals of Solving Optimization Problems

The SVM optimization problem is a case of continuous optimization. An optimization problem is one that has an objective function that needs to be maximized or minimized.

There are two types of optimization problems: unconstrained and constrained. In an unconstrained function, there is only objective function and no constraints. Here, the objective function is differentiable and one has access to the gradients of each points in space, and hence, one can find maximum or minimum of the objective function.

The SVM optimization problem is a constrained optimization problem as there are many constraints. The goal of the optimization problem is to get optimal w and b . So, the tasks required are – formulation of the optimization problem, finding the gradient with respect to the parameters and finding optimal parameters that minimize $\|w\|$.

Lagrangian method is required to convert the constrained optimization problem to unconstrained optimization problem. A minimization problem can be stated as follows:

$$\begin{aligned} &\text{Min } f(x) \\ &\text{Subject to } g(x) \leq 0 \end{aligned} \quad (11.21)$$

Lagrangian multipliers are used to solve the problems. Lagrangian can be introduced in Eq. (11.21) as follows:

$$\mathcal{L}(x, \alpha) = f(x) - \alpha^T g(x)$$

(Continued)

This can be written as:

$$\begin{aligned} \min_x \max_{\alpha} \mathcal{L}(x, \alpha) \\ \text{subject to } \alpha \geq 0 \end{aligned} \quad (11.22)$$

The parameters w, b are dependant of α . So, the α value should be optimal. But Lagrangian function works only for equality constraints. Since SVM has Lagrangian function with inequality constraints, KKT conditions are used to obtain the values of optimal Lagrangian multipliers.

It is better to convert a minimization problem to a maximization problem; and for this dual form of the optimization problem is preferred. What is a dual form? An optimization problem may have two views – primal and dual. If the primal optimization problem does not yield any solution one can opt for the dual optimization problem that is guaranteed to give solution. As in this case, the primal optimization problem does not result in solution. One opts for dual formation where these tasks are done, that is, the problem is formulated and values of primal optimization are substituted. The loss function is simplified, and finally, the optimal values of α are obtained and maximized.

If an optimization problem is a minimization problem, then the dual problem will be a maximization problem. If a solution is obtained for the dual problem, then it will be the lower bound of the minimization problem. In short, the solution of the minimization problem will be less than or equal to the solution of the primal minimization problem. To reduce the number of parameters, the Lagrangian can be transformed into a function of Lagrangian multipliers only. The dual formulation of the Lagrangian function is done by swapping the conditions as:

$$\begin{aligned} \min_{\alpha} \max_x \mathcal{L}(x, \alpha) \\ \text{subject to } \alpha \geq 0 \end{aligned}$$

The aim is to start with x in the primal problem and α as the starting point in dual problem to close the gap between primal and dual problems. The dual optimization problem yields the results and helps in finding optimal Lagrangian multipliers which together provide the optimal values of w and b .

Sequential Mini Optimization (SMO) also can be used to solve optimization problems.

The KKT conditions are given below:

1. $\nabla_w \mathcal{L}(w, b, \alpha) = 0$ and $\frac{\partial \mathcal{L}}{\partial b} = -\sum_{i=1}^n \alpha_i y_i = 0$ (Stationary Condition)
2. $\alpha_i \geq 0$ (Dual Feasibility Condition)
3. $\alpha_i (y_i (wx_i + b) - 1) = 0$ (Complementary Slackness condition)
4. $y_i (wx_i + b) - 1 \geq 0$ (Primal feasible Condition) (11.23)

As per the KKT conditions, one can take the derivative function with respect to the vector w and set it to zero as below:

$$\begin{aligned} \nabla_w &= w - \sum_{i=1}^n \alpha_i y_i x_i = 0 \\ \therefore w &= \sum_{i=1}^n \alpha_i y_i x_i \end{aligned}$$

And taking the derivative with respect to b and setting it zero gives:

$$\frac{\partial \mathcal{L}(w, b, \alpha)}{\partial b} = -\sum_{i=1}^n \alpha_i y_i = 0$$

One can substitute this back to get the Eq. (11.24):

$$\begin{aligned} \max \mathcal{L}(\alpha) &= \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j x_i \cdot x_j \\ \text{Subject to } \alpha_i &\geq 0, \text{ for } \forall_i i = 1 \dots n \text{ and} \\ \sum_{i=1}^n \alpha_i y_i &= 0 \end{aligned} \quad (11.24)$$

This is called Wolfe dual problem of the optimization problem.

Dual Lagrangian Problem

Dual problem involves only Lagrangian multipliers and the training data (no w and B), unlike the original problem where parameters such as w and b were present. Since, the solutions are equivalent, dual form is a better choice. Also, the minus sign in the objective function indicates that the problem is now maximization instead of minimization. Many feel comfortable with maximum problems and hence this conversion of primal to dual is favoured.

The dual Lagrangian objective function can be given as Eq. (11.25).

$$\begin{aligned} \max \mathcal{L}(\alpha) &= \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j x_i \cdot x_j \\ \text{Subject to } \alpha_i &\geq 0, \text{ for } \forall_i i = 1 \dots n \text{ and} \\ \sum_{i=1}^n \alpha_i y_i &= 0 \end{aligned} \quad (11.25)$$

From the Lagrangian multipliers, α_i , one can find w and b as follows:

$$\begin{aligned} w &= \sum_{\alpha_i > 0} \alpha_i y_i x_i, \text{ and} \\ y_i (w x_i + b) &= 1 \end{aligned}$$

This implies:

$$b = \text{avg}_{\alpha_i > 0} \frac{1}{y_i} - w \cdot x_i \quad (11.26)$$

Or alternatively, this can be given as:

$$b = \frac{1}{S} \sum_{i=1}^S (y_i - w \cdot x_i). \text{ Here, } S \text{ is the number of support vectors.}$$

One can compute b as the average of the bias of all the datapoints using Eq. (11.26) as $\text{avg}_{\alpha_i > 0} \{b_i\}$. Combining w and b , one can get the hyperplane equation as:

$$h(x) = w \cdot x + b \quad (11.27)$$

The new point x can be predicted as class based on the sign of $h(x)$ in Eq. (11.27) as $\hat{y}_i = \text{sign}(h(x))$.

11.5 SOFT MARGIN SUPPORT VECTOR MACHINES

Recollect the two constraints discussed in the last section. The constraints are given here again for continuity:

1. The decision boundary should be far away from the data points, and hence, the distance should be maximized between the line and the nearest data point. In short, this constraint concerns more about the margin and hence deals with distance error.
2. To avoid misclassifications of data, hence, dealing with classification error.

In SVM, the error is the sum of distance error and classification error.

The real-world problems are complex and often datasets are severely affected by noise and outliers. So, sometimes, the imposition of the constraint 1 can cause problems and may lead to more misclassifications. This is called distance error. In this case, the constraint 1 can be relaxed to avoid misclassification during generalization. Figure 11.7 (a) shows the effect of outliers. Even though the margins are wide, many samples are present between the lines leading to misclassification. On the other hand, closer lines have no misclassifications.

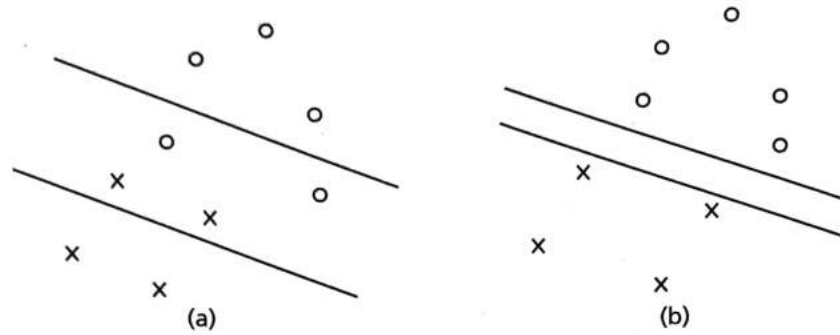


Figure 11.7: (a) Wide Margin but with Misclassification Due to Outliers
(b) Small Margin but with Correct Classification

The misclassifications can be avoided by relaxing criteria 1, that is, by reducing the margin as shown in Figure 11.7 (b). This relaxation of constraint results in 'Soft Margin', where the focus is to reduce misclassification of distance. The second constraint indicates that there should not be any misclassification. This concerns about the classification error. Therefore, the error in SVM is the sum of distance error due to constraint 1 and classification error of constraint 2.

Soft margin SVM allows us to control the tradeoff between the wide margin of decision boundary and classification error through a parameter called C . In all other aspects, SVM is same as the previously discussed 'Hard margin SVM'.

The new objective function is given as:

$$\min_{w, b, \xi_i} \left\{ \frac{\|w\|^2}{2} + C \sum_{i=1}^n (\xi_i) \right\} \quad (11.28)$$

This objective function is subjected to the constraints:

$$y_i(w \cdot x_i + b) \geq 1 - \xi_i \text{ for all the points of } x \text{ in the dataset and} \\ \xi_i \geq 0, \forall x_i \in D$$

Here, ξ represents slack variables, C is the regularization constant, and k is a parameter that determines the error loss functions. It can be observed that the objective function has margin component same as Hard SVM and C times the classification error component. Recollect the fact that errors in SVM are distance error and classification error.

Let us discuss the roles of these parameters now:

Here, C is the regularization constant. As said earlier, this is an important parameter that provides the tradeoff. When C is large, the classifier error is dominated by the classification error and hence focus of SVM is on classification rather than the margin. If C is small, then the formula is dominated by distance error and hence the focus of SVM is more on the margin. If $C = 1$, then the focus is on both margin as well as classification errors. Thus, the parameter C controls the closeness and fairness of the decision boundary depending on the situations based on the given problem and dataset.

ξ_i is called a slack variable. Slack variables are used to denote error vector by assigning a low influence on the points that cause error. Slack variables are used in optimization problems as a measure of the number of misclassifications. Finally, the objective is to minimize the slack variables to minimize the error. The soft margin SVM classifier aims to maximize the margin and at the same time allows data points to violate the constraints according to the slack variables. The errors of the misclassified samples are shown in Figure 11.8 where the error is shown as ' e '.

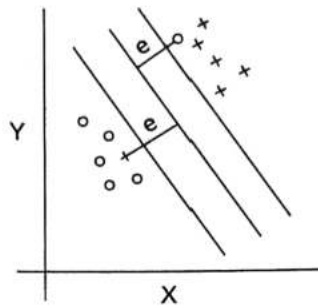


Figure 11.8: Illustration of Error Due to Classification

If $0 < \xi_i < 1$, the point ξ_i is within the margin and still correctly classified. If $\xi_i \geq 1$, the point ξ_i is misclassified.

One more parameter in the above equation is k . If $k = 1$, then the loss is called hinge loss and if $k = 2$, the loss is called quadratic loss. These are discussed in Box 11.3.

Box 11.3: Hinge Loss

Hinge loss (or Hinge loss function) can be used for SVM models for binary classification where the target is $\{+1, -1\}$. If the prediction of SVM is ± 1 and the actual classification score is y , then the hinge loss $l(y)$ is given as:

$$l(y) = \max(0, 1 - y_i(w \cdot x + b)) \quad (11.29)$$

Hinge loss penalizes the wrong decision and right predictions that are not confident. If the prediction and actual values' signs are opposite, then $l(y)$ increases linearly with y in Eq. (11.29) and $(y) < 1$ if the signs match. Hence, collective hinge loss of the dataset should be minimized. Sometimes, squared hinge loss is used. Squared hinge loss is the square of the score hinge loss.

When $k = 2$, quadratic loss is used in the optimization problem. Mean Square Error (MSE) is another measure that is average of the squared difference between predictions and actual observations. In this case, the deviated samples are penalized much.

Hinge loss is preferred over quadratic loss as it yields SVM that is immune to noise. Also, it removes noisy attributes automatically to improve accuracy. In other words, the method has an inherent feature selection capability. Hence, hinge loss is preferred over quadratic loss.

The rest of the implementation of soft margin SVM is same as the hard margin SVM.

As usual, the primal Lagrangian optimization can now be formulated as:

$$\mathcal{L} = \frac{1}{2}\|w\|^2 + C \sum_{i=1}^n \xi_i - \sum_{i=1}^n \alpha_i \left(y_i (w \cdot x_i + b) - 1 + \xi_i - \sum_{i=1}^n \beta_i \xi_i \right) \quad (11.30)$$

Here, α_i and β_i are independent undetermined Lagrangian coefficients. Since the constraints involved are inequality constraints, like hard-margin SVM classifiers, KKT conditions are used. Using the KKT conditions, one can compute w and b using the partial derivatives as in the hard-margin SVM classifier. KKT conditions are discussed in section 11.4.1.

Now, differentiating the Lagrange function with respect to KKT conditions of Eq. (11.23), one gets:

$$\begin{aligned} \nabla_w \mathcal{L} &= w - \sum_{i=1}^n \alpha_i y_i x_i \text{ and} \\ \therefore w &= \sum_{\alpha_i > 0} \alpha_i y_i x_i \text{ and} \end{aligned} \quad (11.31)$$

Differentiating with respect to b and equating it to zero, one gets:

$$\frac{\partial \mathcal{L}}{\partial b} = -\sum_{\alpha_i} \alpha_i y_i = 0$$

And differentiating with respect to slack variables, one gets:

$$\begin{aligned} \frac{\partial \mathcal{L}}{\partial \xi_i} &= C - \alpha_i - \beta_i = 0 \\ \therefore C &= \alpha_i + \beta_i \end{aligned} \quad (11.32)$$

This can be substituted back to get the dual formulation and the dual Lagrangian objective function is given as follows:

$$\max_{\alpha} \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j x_i \cdot x_j \quad (11.33)$$

Subjected to the constraints:

$$0 \leq \alpha_i \leq C, x_i \in D \text{ and } \sum_{i=1}^n \alpha_i y_i = 0.$$

This is same as the hard-margin SVM classifiers. The only difference is the additional constraint, $C = \alpha_i + \beta_i$, that is, $\alpha + \beta = C$. Since, $\alpha, \beta \geq 0$, C serves as the upper bound. This is called box constraint. The value of C should be selected properly. So, there should be a trade-off between the accuracy of data fit and regularization. Optimal choice of C depends on the nature of dataset and given problem. The optimal value of C can be found from cross-validation.

For a new point z , one can compute using Eq. (11.34) as follows:

$$\hat{y} = \text{sign}(w \cdot z + b) \quad (11.34)$$

$$\text{where, } w = \sum_{i=1}^n \alpha_i y_i x_i \text{ and } b = \text{avg}_{\alpha_i > 0} \left(\frac{1}{y_i} - w x_i \right).$$

11.6 INTRODUCTION TO KERNELS AND NON-LINEAR SVM

In machine learning applications, the data can be text, image, sequence, or video. So, there is a need to extract features from these data prior to classification. Hence, in the real world, many

classification models are complex and mostly require non-linear hyperplanes. Consider the Figure 11.9 where the samples cannot be separated by a linear hyperplane and require a non-linear hyperplane. One solution is to map the data into a higher-dimensional space and define a separating hyperplane.

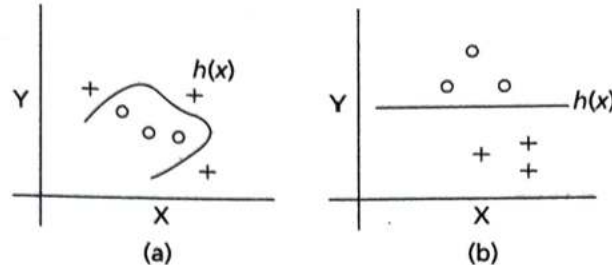


Figure 11.9: (a) Non-Linear Hyperplane (b) Need for Mapping

The mapping process, denoted as $\phi(x)$, is the vector representation of the feature x . In short, mapping transforms a data point in the input space and maps to another point in a space called feature space. Normally, mapping functions are used to map data from a lower dimension to a higher dimension. In Figure 11.9 (b), it can be observed that when the data is mapped from 2D to another feature space, the data points are nicely segregated in different planes and hence can be separated by a plane. So, mapping functions play an important role in non-linear classification.

For example, one mapping function $\phi: \mathbb{R}^2 \rightarrow \mathbb{R}^3$ used to transform a 2D data to 3D data is given as follows:

$$\phi(x, y) = (x^2, \sqrt{2}xy, y^2) \quad (11.35)$$

Example 11.3: Consider a point (2, 3) and apply the mapping $\phi(x, y) = (x^2, \sqrt{2}xy, y^2)$ to get a 3D data.

Solution: One can get 3D data by plugging in the values as $x = 2$ and $y = 3$ in Eq. (11.35) as: $\phi(2, 3) = (2^2, \sqrt{2} \times 2 \times 3, 3^2) = (4, 6\sqrt{2}, 9)$.

While mapping functions play an important role, there are many disadvantages, as mapping involves more computations and learning costs. Also, the disadvantages of transformations are that there is no generalized thumb rule available describing what transformations should be applied and if the data is large, mapping process takes huge amount of time.

In this context, only kernels are useful. Kernels are used to compute the value without transforming the data. What is a Kernel? Kernels are a set of functions used to transform data from lower dimension to higher dimension and to manipulate data using dot product at higher dimensions.

The use of kernels is to apply transformation to data and perform classification at the higher dimension as shown in Figure 11.10.

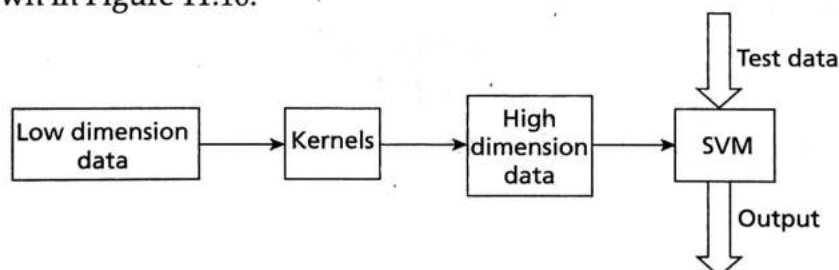


Figure 11.10: Role of Kernels

Kernels are of different types, as discussed below.

Linear Kernel

Linear kernels are of the type $k(x, y) = x^T y$ where x and y are two vectors. Therefore,
 $k(x, y) = \phi(x) \phi(y) = x^T y$ (11.36)

Polynomial Kernel

Polynomial kernels are of the type:

$$k(x, y) = (x^T \cdot y)^q. \quad (11.37)$$

This is called homogeneous kernel. Here, q is the degree of the polynomial. If $q = 2$, then it is called quadratic kernel. For inhomogeneous kernels, this is given as:

$$k(x, y) = (c + x^T \cdot y)^q. \quad (11.38)$$

Here, c is a constant and d is the degree of the polynomial.

If c is zero and degree is one, the polynomial kernel is reduced to a linear kernel. The value of degree d should be optimal as more degree may lead to overfitting.

Example 11.4: Consider two data points $x = \begin{pmatrix} 1 \\ 2 \end{pmatrix}$ and $y = (2, 3)$. Apply homogeneous and inhomogeneous kernels $k(x, y) = (x^T \cdot y)^q$.

Solution: For this, if $q = 1$ in Eq. (11.37), one gets a linear kernel. If $q = 2$, then the resultant of Eq. (11.37) is called a quadratic kernel.

Thus, the linear kernel is given by substituting $q = 1$ in Eq. (11.37) as:

$$\begin{aligned} k(x, y) &= x^T \cdot y \\ &= \begin{pmatrix} 1 \\ 2 \end{pmatrix}^T \cdot (2, 3) \\ &= 8 \end{aligned}$$

The inhomogeneous kernel is given by substituting the value of c as 1 and x in Eq. (11.38) to get,

$$\begin{aligned} k(x, y) &= (1 + x^T \cdot y) \\ &= \left(1 + \begin{pmatrix} 1 \\ 2 \end{pmatrix}^T (2, 3) \right) \\ &= (1 + 8) = 9 \end{aligned}$$

If $q = 2$, then the kernels are called quadratic kernels. The quadratic kernel based on the result of the linear kernel is given as follows:

$$\begin{aligned} k(x, y) &= (x^T \cdot y)^2 \\ &= (8)^2 = 64 \end{aligned}$$

And the inhomogeneous kernel is given as:

$$\begin{aligned} k(x, y) &= (1 + x^T \cdot y)^2 \\ &= (9)^2 = 81 \end{aligned}$$

Gaussian Kernel

RBFs or Gaussian kernels are extremely useful in SVM. RBF stands for radial basis functions. The RBF function is shown as below:

$$k(x, y) = \exp\left(-\frac{\|x - y\|^2}{2\sigma^2}\right) \quad (11.39)$$

Here, γ is an important parameter. If γ is small, then the RBF is similar to linear SVM and if γ is large, then the kernel is influenced by more support vectors. The RBF performs the dot product performed in $\mathbb{R}^\infty$, and therefore, it is highly effective in separating the classes and is often used.

Example 11.5: Consider two data points $x = (1, 2)$ and $y = (2, 3)$. Apply RBF kernel and find the value of RBF kernel for these points.

Solution: Substitute the value of x and y in RBF kernel as given in Eq. (11.39). The squared distance between the points $(1, 2)$ and $(2, 3)$ is given as:

$$(1 - 2)^2 + (2 - 3)^2 = 2$$

If $\sigma = 1$, then $k(x, y) = \exp\{-2/2\} = \exp\{-1\} = 0.3679$.

Sigmoid Kernel

The sigmoid kernel is given as:

$$k(x_i, x_j) = \tanh(kx_i x_j - \delta) \quad (11.40)$$

Kernel Operations

Every kernel operation has an equivalent mapping function. For example, the norm of the vector can be performed as:

$$k(x, x) = x^T x \quad (11.41)$$

Example 11.6: Find the norm of the point $(1, 2)$ and find the distance between the points $(1, 2)$ and $(2, 3)$.

Solution: The norm of the point $(1, 2)$ is given by conventional means as $\sqrt{1^2 + 2^2} = \sqrt{5}$. By using Eq. (11.41), one gets:

$$k(1, 2) = \sqrt{k(x, x)} = \sqrt{\begin{pmatrix} 1 \\ 2 \end{pmatrix}^T \begin{pmatrix} 1 \\ 2 \end{pmatrix}} = \sqrt{5}.$$

Kernel Trick

Kernel trick means replacing the dot product in mapping functions with a kernel function. For example, the kernel $k(x, y)$ given in Eq. (11.42) below and mapping functions are the same.

$$k(x, y) = \phi(x) \cdot \phi(y) \quad (11.42)$$

As usual, kernels help as mapping functions in the non-linear separation of samples and in mapping data from input space to higher-dimensional feature space with least computations.

The gain is done by replacing the dot product of the mapping function with the kernel operation, as performing the kernel operation is much easier. This is illustrated in the following numerical example.

Example 11.7: Consider two data points (1, 2) and (2, 3) Apply a polynomial kernel $k(x, y) = (x^T y)^2$ and show that it is equivalent to mapping function $\phi = (x^2, y^2, \sqrt{2}xy)$.

Solution: The mapping function is given as $\phi = (x^2, y^2, \sqrt{2}xy)$. (11.43)

Let us apply the mapping function first for the first data point (1, 2) using Eq. (11.44) given as:

$$\phi(x) = (1^2, 2^2, \sqrt{2} \times 1 \times 2) = (1, 4, 2\sqrt{2})$$

For the second data point (2, 3) the mapping is given as per Eq. (11.43):

$$\phi(y) = (3^2, 4^2, \sqrt{2} \times 3 \times 4) = (9, 16, 12\sqrt{2})$$

Now the mapping function $\phi(x)\phi(y)$ yields:

$$\phi(x)\phi(y) = (1, 4, 2\sqrt{2}) \begin{pmatrix} 9 \\ 16 \\ 12\sqrt{2} \end{pmatrix} = (1 \times 9 + 4 \times 16 + 24(2)) = 121$$

It can be seen the computation involves many multiplication operations. The operation can be computed quickly using kernel functions. Now, using polynomial kernel function, $k(x, y) = (x^T y)^2$, it can be computed as:

$$\left(\begin{pmatrix} 1 \\ 2 \end{pmatrix} \cdot \begin{pmatrix} 3 & 4 \end{pmatrix} \right)^2 = 11^2 = 121$$

It can be observed that kernel operations are easy as compared to plain mapping functions. Therefore, kernel trick can be applied to replace the dot product operations of mapping functions with the kernels. The clever selection of kernels can solve the problem of non-linearity. This is the reason why kernels are referred to as measures of similarity.

11.7 KERNEL-BASED NON-LINEAR CLASSIFIER

The objective function of kernel-based SVM can be written now as follows:

$$\max_{\alpha} \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j k(x_i, x_j)$$

Subjected to the constraints:

$$0 \leq \alpha_i \leq C \quad \forall_i = 1..n$$

$$\text{And, } \sum_{i=1}^n \alpha_i y_i = 0 \quad (11.44)$$

The weight w can be derived as discussed earlier in terms of α from KKT conditions as:

$$w = \sum_{i=1}^n \alpha_i y_i x_i \quad (11.45)$$

And the classification rule for a new point z is given as:

$$\begin{aligned} y &= \text{sign}(w \cdot z + b) \\ &= \text{sign}\left(\sum_{i=1}^S \alpha_i y_i x_i z + b\right) \end{aligned} \quad (11.46)$$

Here, S is the set of support vectors, α_i is the Lagrangian multiplier, y_i is the class label, b is the bias and z is the new point to be classified.

The parameter ' b ' can be calculated for support vectors $\alpha_i > 0$ and slack variables of zero. These points, say j , are only used to compute b as:

$$\begin{aligned} b &= y_j - wx_j \\ &= y_j - \sum_{i=1}^n \alpha_i y_i x_i x_j \end{aligned} \quad (11.47)$$

The decision boundary using the kernel is given as:

$$\hat{y} = \text{sign}\left(\sum_{i=1}^n \alpha_i y_i k(x_i \cdot z) + \left(y_j - \sum_{i=1}^n \alpha_i y_i k(x_i, x_j)\right)\right) \quad (11.48)$$

Here, z is the new data and x_i is the support vector found during the training period and x_j is the test vector. y_i is the class label and b is the bias.

Since, the computation is done for only a set of support vectors, the SVM can also be called as sparse kernel machine.

Extending Binary Classifier to Multiclass Problems

The binary classifier can be extended for a multi class classifier too if one category is split out and all other categories are merged, and also by using the strategy of one against one, where $k(k-1)/2$ models are constructed. Here, k is the number of categories.

11.8 SUPPORT VECTOR REGRESSION

SVM is mostly used for classification problems. But it can be used for regression problems too. In classification problems, the outputs are discrete values $y \in \{+1, -1\}$. But, in regression problems, the output is a continuous value $y \in \mathbb{R}$. The regression problem can be stated as below, given the dataset:

$D = \{(x_1, y_1), (x_2, y_2) \dots (x_n, y_n)\}$, $x \in \mathbb{R}^n$, $y \in \mathbb{R}$, where x is an n -dimensional datapoint.

The focus is to learn the input and output relation as a function $\hat{y} = f(x)$. This is given as SVR as $f(x) = w \cdot x + b$. Like linear regression finds residuals and minimizes it, SVM regression aims to minimize ϵ -insensitive loss.

Thus, the goal of SVR is to find a function $\hat{y} = f(x)$ that has at most deviation of ϵ from the actual values. If the error is within the range, then it does not matter, else it is considered an error. The error insensitive region is also known as ϵ -tube. As said earlier, within ϵ -tube, the errors

are tolerated. If the error is above or below the ε - tube, the errors are considered linearly. This means that the deviation is more if the point is far away from the ε - tube. The parameter ε is more important. If ε is more, then there will be a reduction in accuracy. If ε is low, then it results in overfitting.

This is shown in Figure 11.11.

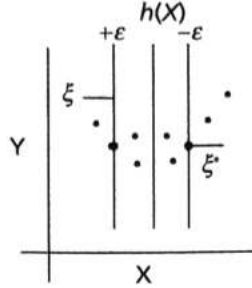


Figure 11.11: Support Vector Regression

In SVR, ε - insensitive loss function is given as follows:

$$\varepsilon - \text{insensitive loss function} = \begin{cases} 0 & \text{if } |y - f(x)| \leq \varepsilon \\ |y - f(x)| - \varepsilon & \text{otherwise} \end{cases} \quad (11.49)$$

or ε - tube.

Let us introduce slack variables ξ, ξ^* . If the datapoints fall within the ε - tube, the slack variables are zero. If the data point is above ε - tube, then $|y_i - x_i| - \varepsilon = \xi_i$ and if the point is below, then $|y_i - x_i| - \varepsilon = \xi_i^*$. Support vector regression minimizes the margin error (distance error) and classification error functions.

The corresponding SVR optimization problem can be stated as follows:

$$\min \frac{1}{2} \|w\|^2 \quad (11.50)$$

Subjected to the constraint $|y_i - w_i x_i| \leq \varepsilon$.

One can add slack variables in Eq. (11.51) that indicate the deviation of a data point that falls outside the margin. The modified optimization objective function is given as:

$$\min \frac{1}{2} \|w\|^2 + C \sum_{i=1}^n \xi_i + \xi_i^*$$

Subjected to the constraints:

$$\begin{aligned} y_i - w_i x_i - b &\leq \xi + \xi_i^* \\ w_i x_i + b - y_i &\leq \xi + \xi_i^* \\ \xi_i, \xi_i^* &\geq 0 \end{aligned} \quad (11.51)$$

The solution is again that the hard-margin SVM is given by Lagrangian dual problems. This problem uses a linear loss function. The other loss functions that can be considered are Huber loss and quadratic loss.

The idea of SVR now is to construct a hyperplane (w, x_i) and boundary for $(w, x_i) + \varepsilon$ and $(w, x_i) - \varepsilon$. The lines can be controlled by a parameter C. If C is zero, then the points are collapsed into one. If C is not zero, then the tolerance of points outside of ε is increased. The choice of C plays an important role and grid searched to obtain a better solution. So, the parameters that need to be tuned are C, parameters of the kernel and ε .

The Lagrangian and dual Lagrangian functions are constructed as hard-margin SVM. Applying the KKT conditions and finding partial derivatives of dual Lagrangian and equating it zero results in:

$$w = \sum_{i=1}^n (\alpha - \alpha_i^*) x_i \quad (11.52)$$

It can be observed that w is a linear combination of training values x_i . Data points inside the ε -tubes will have Lagrangian multipliers at zero.

$$\begin{aligned} (C - \alpha_i) \xi_i &= 0 \\ (C - \alpha_i^*) \xi_i^* &= 0 \end{aligned} \quad (11.53)$$

This condition ensures that the only samples that correspond $\alpha_i, \alpha_i^* = C$ lie outside the ε -insensitive tube. For those points that fall within ε -insensitive tube, ξ_i, ξ_i^* are zero and α_i, α_i^* would be in the range $(0, C)$.

The weight w can be computed as:

$$\begin{aligned} b &= y_i - w \cdot x - \varepsilon \text{ for } \alpha_i \in (0, C) \\ b &= y_i - w \cdot x + \varepsilon \text{ for } \alpha_i \in (0, C) \end{aligned} \quad (11.54)$$

For all points, $\alpha_i, \alpha_i^* \in (0, C)$ is used to compute ' b ' and the average of all ' b ' is the final ' b ' value. Finally, the fitted line can be desired as the weighted sum of support vectors. Finally, a linear SVR can be constructed as:

$$y = \sum_{i=1}^s (\alpha_i - \alpha_i^*) x_i' x_j + b \quad (11.55)$$

Here, s is the set of support vectors, α_i is the Lagrangian multiplier, x_i is the support vector found during the training period, x_j is the test vector and b is the bias.

The non-linear SVR in terms of kernel function is given as:

$$y = \sum_{i=1}^s (\alpha_i - \alpha_i^*) K(x_i, x_j) + b \quad (11.56)$$

Here, s is the set of support vectors, α_i is the Lagrangian multiplier, x_i is the support vector found during the training period, x_j is the test vector and b is the bias.

11.8.1 Relevance Vector Machines

Relevance Vector Machines (RVM) is a new development that captures the imagination of machine learning experts. RVM is a Bayesian sparse linear model that uses Bayesian inference. RVM can be used for both regression and probability classification.

RVM uses a kernel like Gaussian process model as given in Eq. (11.57).

$$k(x, x') = \sum_{j=1}^n \frac{1}{\alpha_j} \phi(x, x_j) \phi(x', x_j) \quad (11.57)$$

Here, ϕ is a kernel function, $x_1, x_2, \dots, x_n$ are input vectors of the training set, and α_j are the variances of the prior on the weight vector w . The parameters of the RVM can be initialized using the EM algorithm.

The RVM can generate decision functions that are sparser as compared to the SVM. Its support vectors are also small compared to that of SVM. It also does not have a regularization parameter C . The RVM is useful in many pattern recognition tasks like classification and regression.

Summary

- SVM is a supervised learning algorithm that uses a labelled data as input and learns learning functions.
- The aim of SVM is to produce a decision plane that defines the boundary between the classes. The focus is to maximize the margins as much as possible.
- The maximal margin problem can be formulated as an optimization problem.
- The convex optimization problem can be solved using Lagrangian multipliers.
- The dual problem can be formulated. Its solution is $\leq$ the solution of the primal problem.
- Soft margin SVM can be used to solve non-linearly separable problems.
- The optimization problem can be solved by introducing slack variables.
- Binary classification can be extended for multiclass classifiers by including one category and when all other categories are merged. Similarly, the multiclass classifiers can be constructed by constructing $k(k-1)/2$ models, where ' k ' represents categories.
- One way of transforming a low-dimensional data to a high-dimensional data is with kernels. Kernels are functions that perform dot product in some other higher dimension.
- Types of kernels are linear kernel, polynomial kernel and Gaussian RBF kernels.
- Kernel trick replaces dot product using kernels.
- Kernel-based classifiers can be constructed for non-linear classifications.
- Support vector regression uses ϵ - margin to fit a linear or non-linear line for regression problems.

Key Terms

- **Hyperplane** – A decision plane that defines the boundary between classes and is a set of points that satisfy the hyperplane equation $h(x) = w \cdot x + b = 0$.
- **Linear Separable SVM** – A linear hyperplane that classifies the samples.
- **Margin** – The space between the hyperplane and its boundary lines.
- **Hard Margin** – There is no margin of error by ensuring that the constraints are not violated. If the constraints are violated, SVM creates a soft-margin hyperplane that allows violation of constraints and hence tolerates some error.
- **Functional Margin** – It is a testing function for classification of data points. But it does not indicate whether the point is close or far from the hyperplane. That is indicated by the geometric margin which is a scaled version of functional margin.
- **Optimization Problem** – A problem of an objective function that needs to be maximized or minimized is subjected to a set of constraints. If there are constraints involved, then the problem is called unconstrained optimization.
- **Optimal Solution** – A solution that maximizes or minimizes objective function without violating the constraints.
- **Lagrangian Problem** – A form of solving an optimization problem using Lagrangian multipliers.

- **Dual Problem** – It is a maximization problem that corresponds to a primal Lagrangian minimization problem.
- **Soft Margin Classifiers** – Classifiers used to separate samples that are not linearly separable.
- **Slack Variables** – Dummy variables used to measure the number of misclassifications.
- **Regularization Constant** – Penalty terms introduced as part of objective function.
- **Support Vector Regression** – A regression method for prediction of continuous variables.
- **Transformers** – A function that is used to map low-dimensional data to a higher-dimensional data.
- **Kernels** – Functions that perform dot product in another space.
- **SVR** – Support vector regression method to predict continuous values for a given set of features.
- **Kernel Trick** – The method of replacing a dot product of a mapping function by a kernel to simplify the computations.

Short Questions

1. What is a margin?
2. List out the advantages of SVM.
3. Formulate the optimization problem as a problem of maximal margin.
4. What are the differences between primal and dual problems?
5. List out KKT conditions.
6. What are slack variables?
7. List out the advantages and disadvantages of mapping functions or transformations?
8. List the types of kernels.
9. What is a kernel trick?
10. List out the differences between SVM and Relevance Machines.

Long Questions

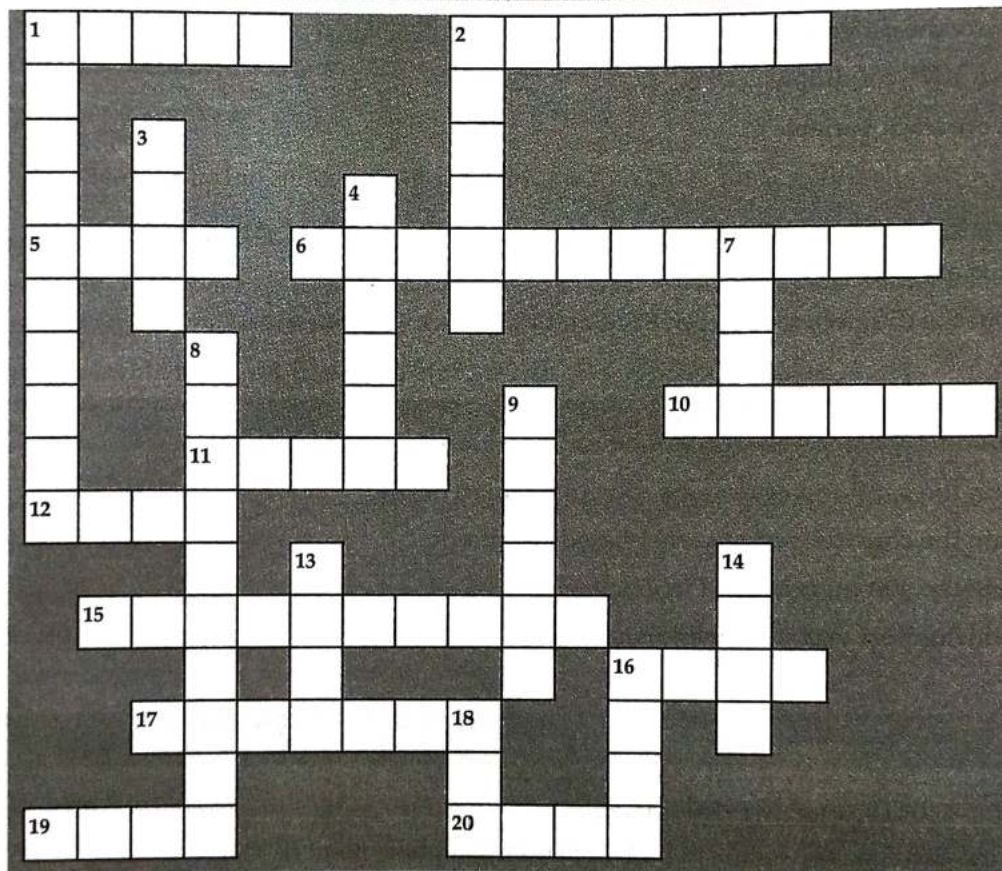
1. Explain the support vector regression problem in detail.
2. Explain the construction of kernel-based classifiers.
3. Explain the soft margin SVM in detail.
4. Explain support vector regression in detail.

Numerical Problems

1. Compute the norm of the vector (2, 8).
2. Find the distance between a point (2, 6) and hyperplane point (2, 4).
3. The hyperplane function for two variables is $b + a_1x_1 + a_2x_2$. If two hyperplanes are given are for classifier 1 as $5 + 2x_1 + 2x_2$ and $3 + 20x_1 + 20x_2$, for classifier 2. Find the distance error function and pick a good classifier constructed using these hyperplanes?

4. For the given points (3, 1), (3, 2) and (4, 0) that belong to class positive and points (1, 0), (0, 1) and (0, -1) that belong to negative class, draw an optimal hyperplane to classify the points.
5. Consider the mapping function $\phi(x_1, x_2) = (x_1^2, \sqrt{2}x_1x_2, x_2^2)$ and map the points (2, 3) and (6, 7).
6. Apply the following transformations for the data points (1, 4) and (2, 7):
 - (a) $\phi : x \rightarrow z : [x, y, x^3 + y^3]$
 - (b) $\phi : x \rightarrow z : [x, y, \sqrt{2}xy]$
7. Find homogeneous and inhomogeneous kernel values for the following data points for a polynomial kernel of degree 1 and 2.
 - (a) $\begin{pmatrix} 1 \\ 2 \end{pmatrix}$
 - (b) $\begin{pmatrix} 5 \\ 6 \end{pmatrix}$
8. Using the RBF kernel, consider the mapping of a point (2, 3) and (5, 7).
9. Using kernel operation, find the norms of the vector:
 - (a) $\begin{pmatrix} 7 \\ 8 \end{pmatrix}$
 - (b) $\begin{pmatrix} 9 \\ 7 \end{pmatrix}$
10. Prove that kernel operations work for the points (1, 4) and (4, 5) for the kernel $k(x, y) = (x^T y)^2$.

Crossword



Across

1. SVR is not used to predict continuous variables. (True/False)
2. The focus of SVM is to produce a _____ (maximal/minimal) margin among the classes.
5. An optimization problem can be stated as a primal or dual optimization problem. (True/False)
6. Dual form problem is a _____ (maximization/minimization) problem.
10. Kernel trick replaces dot product by a _____.
11. A _____ is used to separate classes in 3D.
12. A _____ is used to separate classes in 2D.
15. SVM is a _____ (supervised/unsupervised) algorithm.
16. A solution is called an optimal solution if it satisfies the objective function without violating the constraints. (True/False)
17. SVM uses _____ (labelled/unlabelled) data.
19. An objective function is always in the form of minimization or maximization. (True/False)
20. Dot product that can be replaced by kernels is called as kernel trick. (True/False)

Down

1. A point is closer or farther from the decision plane is not indicated by _____ (functional/geometric) margin.
2. Support vector regression uses epsilon _____ to fit a linear or nonlinear regression.
3. Slack variables are introduced in optimization problems to change sign. (True/False)
4. The space between the hyperplane and its boundary lines is called _____.
7. A lower order function can be transformed to a higher order function by using transformations. (True/False)
8. A decision plane that defines boundary among classes for more than three dimensions is called _____.
9. Lagrangian multipliers are used to solve _____ optimization problems.
13. Kernels transform low dimensional data to high dimensional data. (True/False)
14. Slack variables can be used to measure the number of misclassifications. (True/False)
16. Kernel based classifiers can be used for non-linear classification. (True/False)
18. Kernel functions perform _____ product in higher dimensions.



Chapter 12

Ensemble Learning

"We unfortunately live in a corporate world where group decision making is made to avoid failure rather than to achieve success."

— Bill Cahan

Ensemble learning is a machine learning model which combines the predictions of several machine learning models to improve the prediction accuracy of using a single classifier. In other words, ensemble models have been developed to overcome the limitations of using a single-hypothesis machine learning model. The general principle of this model is to ensemble hypotheses of different models either by combining the collective predictions of different models into a single final model or by employing different models in a sequential fashion to increase the accuracy and robustness of learning. Ensemble models are useful to solve both classification and regression problems where they reduce bias and variance to boost the accuracy of models. They are used in many real-world applications, such as remote sensing, security, gene classification, image segmentation, video retrieval, healthcare, and so on.

Learning Objectives

- Understand the basics of ensemble learning, its advantages, and disadvantages
- Study the different ensembling techniques such as parallel ensemble models, sequential ensemble models and incremental ensemble models
- Gain knowledge about parallel ensemble models like Voting, Bootstrap resampling and Bagging and explore the popular bagging technique called Random Forest
- Learn about incremental ensemble models such as Stacking and Cascading
- Acquire knowledge about sequential ensemble models and explore the popular boosting technique called AdaBoost model

12.1 INTRODUCTION

Ensembling is a process of combining many weak learners to produce a strong learner that performs better than a single weak learner. The idea is to learn better with a set of models, instead of making one model to learn better. It is like getting opinions from many in order to decide and to protect oneself in decision-making.

A very simple way of ensembling is majority voting or taking average or weighted average of predictions of multiple machine learning models or base learners. 'Stacking' is another way of ensembling that uses the predictions of multiple models as 'features' to train a new model and then predict from that model. Another approach is to employ the same machine learning model multiple times with different datasets and combine the hypothesis generated from these models.

Bagging and Boosting are two popular models of ensemble learning. Bagging is a method of ensembling simultaneously many independent models and averaging the predictions obtained from the different models, which in fact, is more robust than the prediction made by a single classifier. Thus, bagging solves the overfitting problem which normally happens with a single classifier. On the other hand, boosting employs different classifiers in a sequential fashion where the prediction made by a classifier is enhanced by boosting the weights for the data instances and consecutively the predicted output is given as input to the next learning model. Thus, boosting increases the accuracy of predictions made by the previous learning model. Bagging minimizes the variance of the learning model, whereas boosting increases the accuracy of prediction.

Ensemble learning is an approach to minimize bias and variance and to build accurate models that avoid over-fitting and under-fitting of learning models. Even, cross-validation can be done to train on many datasets and average their predictions to ensemble different predictions from many models. Fine tuning a machine learning model will improve the accuracy of the model. For example, fine tuning the parameters of the model, which mean building a random forest classifier to tune the number of trees to build and the number of variables to choose for splitting, etc. Ensembling is to combine various fine tuned individual models that improve the performance of the model. On the other hand, there is always a tradeoff between the accuracy and complexity of the ensemble models.

To perform ensemble learning, different subsets of data are generated from the training dataset where each subset contains data instances that are chosen randomly with replacement. These subsets are given as input to different learning models simultaneously in bagging and sequentially in boosting to train the classifier. Thus, ensembling can be done in different ways such as using different machine learning models, different hypothesis, different hyperparameters, different population (data sets), different initial seed and different features, etc.

Some of the advantages and disadvantages of ensemble learning are listed below:

Advantages of Ensemble Learning

1. Minimizes bias and variance of a weak learner
2. Reduces overfitting and underfitting of the model
3. Increases accuracy of prediction
4. Handles higher dimensionality data efficiently

Disadvantages of Ensemble Learning

1. Does not give precise values for the classification and regression model
2. Error prone and sometimes leads to overfitting
3. Requires careful tuning of the number and type of features used

Let us study about all these approaches of ensembling in a detailed manner in the further sections.

12.1.1 Ensembling Techniques

Ensemble models are generally classified as 'Parallel' ensemble models and 'Sequential' ensemble models based on how the base learners are ensembled for training. In a parallel ensemble mode, the base learners learn simultaneously and independently, whereas in a sequential ensemble mode the base learners are used sequentially to learn. Ensembling models can also be classified as 'Homogenous' and 'Heterogenous' ensemble models, depending on the type of base learners used in each stage. If same type of base learner is used for ensembling, then such a model is known as a Homogenous ensemble model, while in case of a heterogenous ensemble model, different type of base learner is used for ensembling.

The parallel ensemble models include Voting, Bootstrap Resampling and Bagging. Random forest model is a popular ensemble model constructed based on the idea of bagging. Some of the sequential ensemble models are AdaBoost, Gradient Tree Boosting and XGBoost which are also called as boosting models. Stacking and Cascading belong to the family of incremental ensemble models. Parallel ensemble learners are time and space efficient, as compared to sequential ensemble learners.

12.2 PARALLEL ENSEMBLE MODELS

Parallel ensemble models provide a simple way of ensembling the predictions from multiple independent models and then applying averaging techniques such as normal average or weighted average or majority voting to estimate the final prediction.

Scan for illustrations of 'Types of Ensemble Models', 'Parallel Ensemble Model', 'Voting', 'Bootstrap Resampling', 'Working Principle of Bagging', 'Learning Mechanism through Stacking', and 'Sequential Ensemble Model'



12.2.1 Voting

It is a simple way of ensembling multiple learners. It is the process of combining multiple machine learning models or using the same model multiple times as base learners and taking average of their predictions in the case of regression problem and considering the maximum vote of their predictions in the case of classification problem.

12.2.2 Bootstrap Resampling

It is a process of creating many bootstrap samples S_i ($1 \leq i \leq M$) using a method called Random Sampling with replacement from the training dataset. Each bootstrap sample contains 63.2% distinct tuples from the original training dataset and the remaining tuples are duplicate tuples drawn from the original dataset. If the training dataset T contains N tuples, M samples can be created, where each bootstrap sample S_i consists of N tuples by randomly choosing N tuples from the original training dataset with replacement.

12.2.3 Bagging

Bagging is otherwise called as Bootstrap Aggregation which combines both bootstrapping and aggregation (Breiman, 1996). It uses Bootstrap Resampling to create M bootstrap samples from the original training dataset T . It then trains M independent weak classifiers with each classifier containing a bootstrap sample S_i ($1 \leq i \leq M$). Then, it combines the predictions made by M weak classifiers either by majority voting if the target variable is a categorical variable or by averaging if the target variable is a continuous variable. By combining or ensembling many classifiers predictions, the model reduces the error or decreases the variance, resulting in the prediction of a single weak classifier.

Algorithm 12.1: Bagging

Input: Training dataset T with N data instances.

Step 1: Create M bootstrap samples S_i ($1 \leq i \leq M$) of N data instances each from the training dataset T using Random sampling with Replacement.

Step 2: Train M weak classifiers H_i ($1 \leq i \leq M$) using the samples S_i ($1 \leq i \leq M$).

Step 3: Combine predictions from all classifiers taking average $H_{ave} = \frac{1}{M} \sum_{i=1}^M H_i$.

12.2.4 Random Forest

This ensemble model is a variant of Bagging which uses decision trees for learning. M bootstrap samples, each of size N tuples, are created by random sampling with replacement from the training dataset of N tuples. These M samples are used to train M decision tree learners. Instead of training different models with the same features, Random Forest chooses a random set of features to construct decision trees. Moreover, the split feature chosen at every node for each decision tree is also different, thus exhibiting a level of differentiation among all trees in the ensemble model. The Random Forest ensemble model uses different hyperparameters that can be tuned prior to learning. Hence, all the decision trees have different leaves but are of the same depth. Then, it uses majority voting or averaging to finalize the prediction from all the decision trees. Thus, Random Forest is a collection of trees with each tree constructed with random samples, random set of features and random split features.

The model improves accuracy and reduces variance by employing more diversity among the different weak learners. Since the learning model is a decision tree with a chosen subset of features, the ensemble model also improves the efficiency with quick searching. Figure 12.1 illustrates the working principle of learning using the Random Forest algorithm.

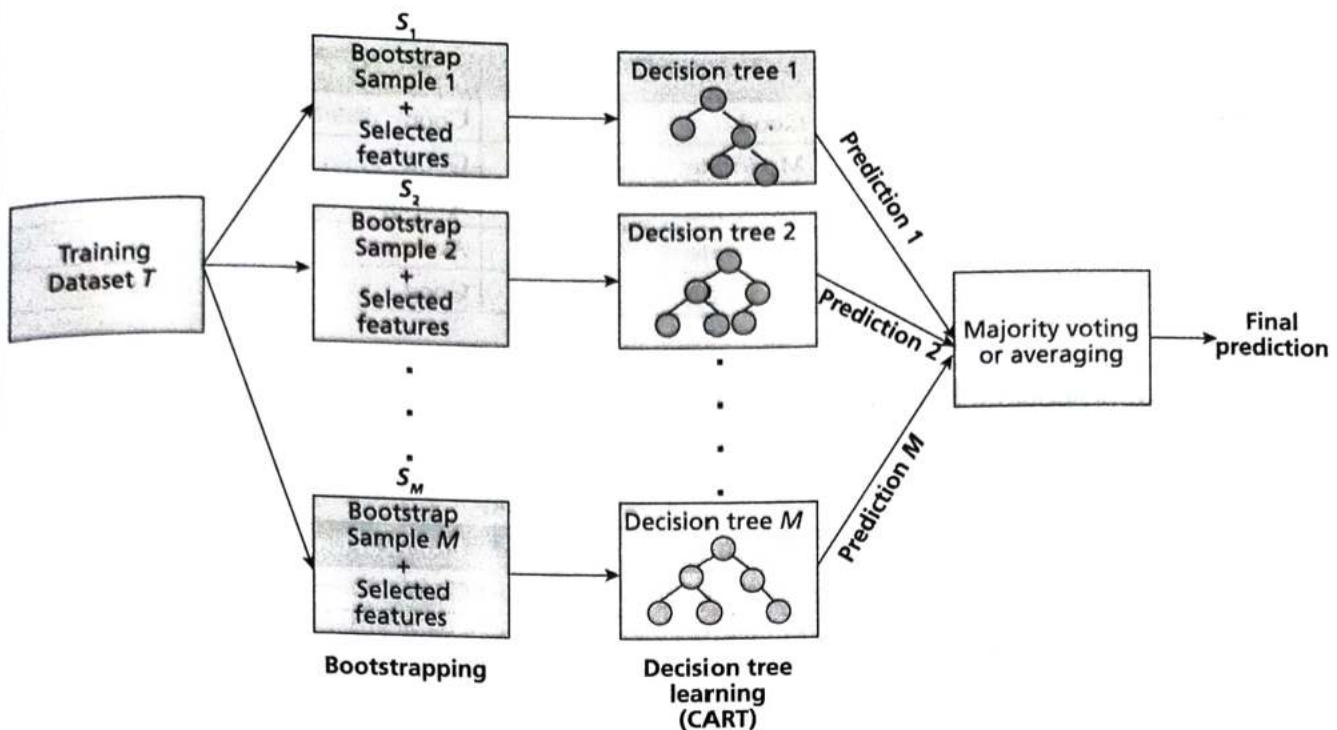


Figure 12.1: Random Forest Model

Algorithm 12.2: Random Forest

Input: Training dataset T with N data instances.

Step 1: Create M bootstrap samples S_i ($1 \leq i \leq M$) of N data instances each from the training dataset T using Random sampling with Replacement.

Step 2: Choose number of Decision trees to be constructed DT_M .

Step 3: Choose the number of features to be used in each tree. Normally, $\sqrt{D}$ randomly chosen features are taken.

Step 4: Choose the depth of the decision tree. All trees are to be same depth.

Step 5: Repeat M times. For each tree:

- (a) Choose a bootstrap sample S_i of N data instances.
- (b) Compute the best split feature to be placed at each node.
- (c) Construct decision tree using CART. Do not prune the tree.

Step 6: Combine predictions from all classifiers taking Average $H_{ave} = \frac{1}{N} \sum_{i=1}^M H_i$ for classification problem or Majority voting in the case of regression problem.

Example 12.1: Consider a training dataset of 5 data instances shown in Table 12.1 which describes the performance of students and their skills obtained during study. Based on their performance and the skills obtained, they would get a Job Offer at the end of the course. Train using a Random Forest model and test this ensemble model.

Table 12.1: Training Dataset

S.No.	CGPA	Interactiveness	Communication Skills	Practical Knowledge	Job Offer
1.	≥ 9	Yes	Good	Good	Yes
2.	< 9	No	Moderate	Good	Yes
3.	≥ 9	No	Moderate	Average	No
4.	< 9	No	Moderate	Average	No
5.	≥ 9	Yes	Moderate	Good	Yes

Solution:

Step 1: Create 3 bootstrap samples of 5 data instances each from the training dataset T using random sampling with replacement as shown in Tables 12.2, 12.3 and 12.4, respectively.

Table 12.2: Sample S_1

S.No.	CGPA	Interactiveness	Communication Skills	Practical Knowledge	Job Offer
1.	≥ 9	Yes	Good	Good	Yes
2.	< 9	No	Moderate	Good	Yes
3.	≥ 9	No	Moderate	Average	No
3.	≥ 9	No	Moderate	Average	No
5.	≥ 9	Yes	Moderate	Good	Yes

Table 12.3: Sample S_2

S.No.	CGPA	Interactiveness	Communication Skills	Practical Knowledge	Job Offer
2.	< 9	No	Moderate	Good	Yes
3.	≥ 9	No	Moderate	Average	No
3.	≥ 9	No	Moderate	Average	No
5.	≥ 9	Yes	Moderate	Good	Yes
5.	≥ 9	Yes	Moderate	Good	Yes

Table 12.4: Sample S_3

S.No.	CGPA	Interactiveness	Communication Skills	Practical Knowledge	Job Offer
1.	≥ 9	Yes	Good	Good	Yes
1.	≥ 9	Yes	Good	Good	Yes
2.	< 9	No	Moderate	Good	Yes
3.	≥ 9	No	Moderate	Average	No
3.	≥ 9	No	Moderate	Average	No

Step 2: Choose number of Decision trees to be constructed as 3.

Step 3: Choose the number of features to be used in each tree. Here, we will choose 2 features to construct every decision tree.

Step 4: Choose the depth of each decision tree to be 2.

Step 5: Repeat the following steps for 3 decision trees that are constructed:

Step 5 (a): Choose bootstrap sample S_1 of 5 data instances to construct Decision tree 1.

Step 5 (b): Compute the best split feature to be placed at each node. Here, we choose CGPA and Interactiveness.

Step 5 (c): Construct decision tree using CART shown in Figure 12.2.

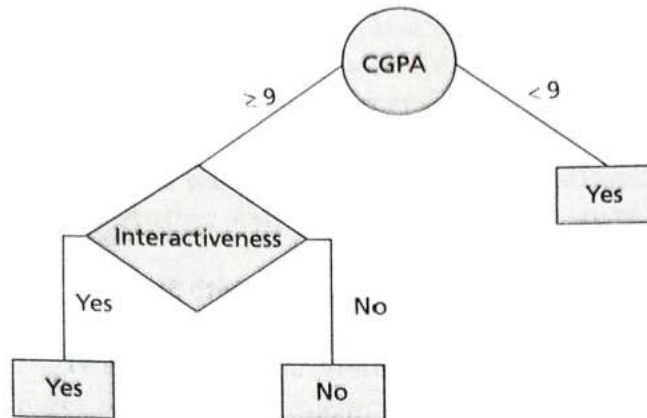


Figure 12.2: Decision Tree 1 with Sample S_1

Step 5 (a): Choose bootstrap sample S_2 of 5 data instances to construct Decision tree 2.

Step 5 (b): Compute the best split feature to be placed at each node. Here, we choose Interactiveness and Practical Knowledge.

Step 5 (c): Construct decision tree using CART shown in Figure 12.3.

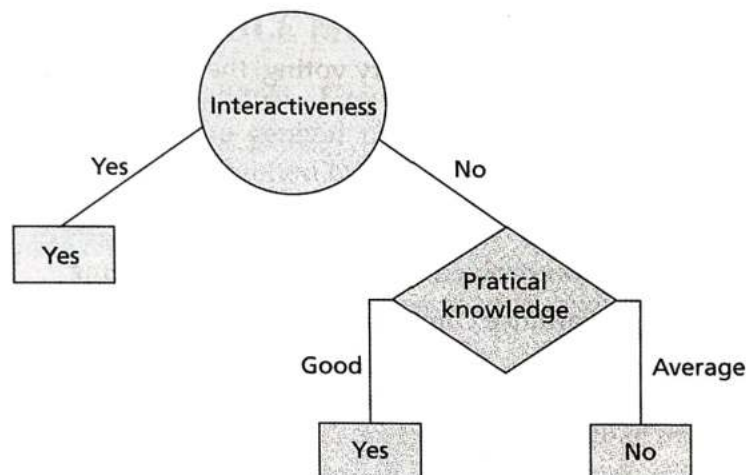
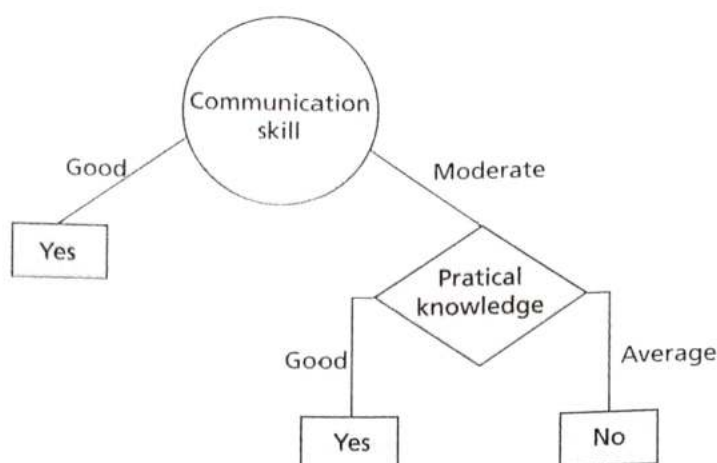


Figure 12.3: Decision Tree 2 with Sample S_2

Step 5 (a): Choose bootstrap sample S_3 of 5 data instances to construct Decision tree 3.

Step 5 (b): Compute the best split feature to be placed at each node. Here, we choose Communication Skills and Practical Knowledge.

Step 5 (c): Construct decision tree using CART shown in Figure 12.4.

Figure 12.4: Decision Tree 3 with Sample S_3

Step 6: Combine predictions from all classifiers taking average $H_{ave} = \frac{1}{N} \sum_{i=1}^M H_i$ for Regression problem or majority voting in the case of classification problem.

Use the data instance 4 as the test instance is not used in the training. The data instance 4 is the Out of Bag instance used for testing.

Following are the predictions given by each of the decision tree constructed using the different samples:

Decision Tree 1 – Yes

Decision Tree 2 – No

Decision Tree 3 – No

It is a classification problem. So, using majority voting, the prediction Job Offer = No is the final prediction of this Random Forest model.

Advantages of the Random Forest Model

1. Can solve both classification problems and regression problems
2. Reduces overfitting and variance compared to a single decision tree
3. Provides more accuracy
4. Performs well even with smaller set of data

Disadvantages of the Random Forest Model

1. Increased complexity in construction and prediction
2. Uses more computational resources

12.3 INCREMENTAL ENSEMBLE MODELS

Incremental ensemble models use the predictions that are learnt from the previous learner as features to train a meta learner and make final predictions. Stacking and cascading models are popular incremental ensemble models which are discussed below.

12.3.1 Stacking

Stacking is a meta modelling technique introduced by Wolpert in the year 1992. Stacking model includes two types of learners called base-learners and a meta-learner. It is a process of ensembling multiple heterogeneous machine learning models or base learners to make predictions and then use these predictions as features to train the second level meta learner model whose prediction is considered as the final prediction. Base-learners use normal machine learning algorithms like Random Forests, SVM, Perceptron, etc., whereas the meta learner fits on the predictions of the base learner. The meta-learner can be meta-regressor or meta-classifier, which collects predictions from each base learner to estimate the final predictions, thereby improving the overall performance of the model. Stacking is making the second level of learner to estimate the biases and correct it. The stacking technique can be further extended to more than two levels which is called as multi-level stacking.

12.3.2 Cascading

A cascading ensemble model is a multistage learning technique with a sequence of classifiers ordered in terms of increasing complexity and specificity such that early classifiers are simple and general whereas later ones are more complex and specific. It moves to the next stage of learning only if the preceding learner's prediction is not confident. In cascaded models, different classifiers are coupled with their input/output function in a cascade, improving performance at each level. In a typical cascading ensemble model, the complexity of the model increases as more models are added to the cascade.

12.4 SEQUENTIAL ENSEMBLE MODELS

Boosting algorithms such as AdaBoost, Gradient Boosting and XGBoost are categorized as Sequential Ensemble models. The general principle of this model is to use multiple weak learners sequentially and the error exhibited by a weak learner is used to give importance to data instances which are further retrained by the next weak learner. Thus, the accuracy of prediction is improved in the ensemble model.

12.4.1 AdaBoost

AdaBoost, which is otherwise called as Adaptive Boosting, is a popular boosting algorithm and it is a non-linear classifier. It is a sequential ensemble model that trains multiple weak classifiers sequentially. It would train one weak classifier at a time, so that the errors made by poorly performing weak classifiers are reduced when they are trained sequentially.

AdaBoost is a greedy algorithm optimizing weights of data instances in the training dataset by adding a weak classifier at each step. Each weak classifier is trained with a random bootstrap sample drawn from the training dataset. Initially, weights are assigned to all data instances in the training dataset with equal probability. If there are 10 data instances in the training dataset, the probability of choosing a data instance to be trained by the first weak classifier would be $1/10$. Hence, equal distribution of weights, that is, $1/10$ is assigned to all data instances. After training by one weak classifier, the weights of all data instances are boosted based on the accuracy of classification done by it. The idea of boosting is to minimize the errors made by a weak classifier

by increasing the weights of misclassified data instances and decreasing the weights of correctly classified data instances. The updated probabilities of data instances are reflected in the training dataset and these instances get trained with the next classifier.

Each weak classifier is also assigned a weight after training, based on the accuracy of its classification. If the classifier is more accurate, it is given more weight. A 50% accurate classifier is given a weight of 0. If the classifier has less than 50% accuracy, it gets a negative weight.

There are many weak classifiers such as Decision trees, Decision stumps, and Multilayer Perceptron. Linear classifiers such as Decision stumps are one-level decision trees which perform poorly with low accuracy. The depth of decision stumps is 1 and it has two leaves. The hypothesis of a Decision Stump model can be given as follows:

$$H_{\text{Decision-Stump}}(x) = \begin{cases} 1 & \text{if } x > \theta \\ 0 & \text{else} \end{cases} \quad (12.1)$$

The final classifier makes a linear combination of all the predictions made by the weak classifiers.

Figure 12.5 illustrates the method of learning using the AdaBoost algorithm.

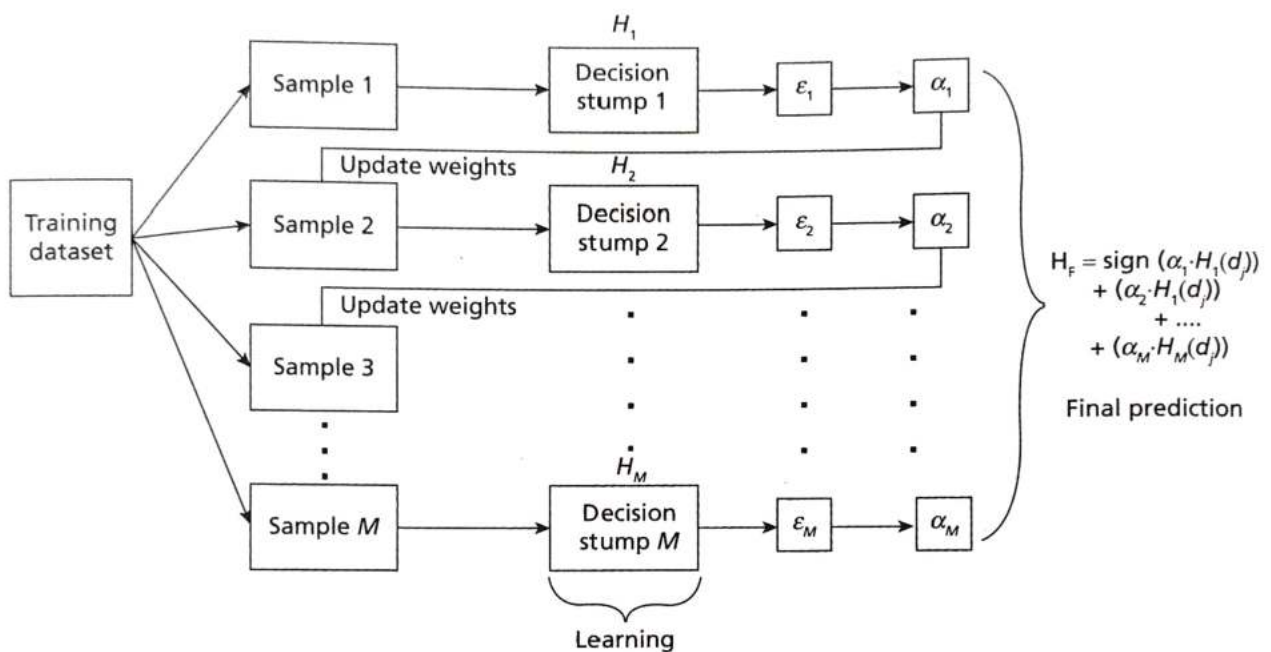


Figure 12.5: Working of AdaBoost Algorithm

Algorithm 12.3: AdaBoost

Input: Training dataset T with N data instances and M weak classifiers.

Step 1: Assign uniform weights $1/N$ to all the data instances d_j ($1 \leq j \leq N$) in the training dataset T , i.e., $wt(d_1) = 1/N$, $wt(d_2) = 1/N$, ..., $wt(d_N) = 1/N$. Here, wt implies weight.

Step 2: Repeat for each weak classifier.

Step 2 (a): Train a weak classifier H_i with a random bootstrap sample from the training dataset T .

(Continued)

Step 2 (b): Compute the weighted error ϵ_i of H_i on current training dataset:

$$\epsilon_i = \sum_{j=1}^N H_i(d_j) wt(d_j)$$

$H_i(d_j) = 0$ if prediction is correct with H_i

$H_i(d_j) = 1$ if prediction is wrong with H_i

Step 2 (c): Compute the weight of each weak classifier:

$$\alpha_i = \frac{1}{2} \ln \left(\frac{1 - \epsilon_i}{\epsilon_i} \right)$$

α_i is based on the error rate.

As the error rate increases, α_i grows exponentially negative.

Better classifiers get exponentially more weight.

If error rate is 50%, the classifier gets a weight of 0.

If error rate is more than 50%, the classifier gets a negative weight.

Step 2 (d): Calculate the normalizing factor Z_i . This is needed so that the sum of the weights of data instances adds up to 1.

Z_i = Total weight of correctly classified instances + Total weight of incorrectly classified instances and Here, wt implies weight.

$$Z_i = wt(\text{correctly classified instance}) \times \text{Number of correct classifications} \times e^{-\alpha_i} \\ + wt(\text{incorrectly classified instance}) \times \text{Number of incorrect classifications} \times e^{+\alpha_i}$$

Step 2 (e): Update the weight of all data instances:

$$wt(d_j)_{i+1} \text{ correct-instance} = \frac{wt(d_j)_i \text{ correct - instance} \times e^{-\alpha_i}}{Z_i}$$

$$wt(d_j)_{i+1} \text{ incorrect-instance} = \frac{wt(d_j)_i \text{ incorrect - instance} \times e^{+\alpha_i}}{Z_i}$$

where, $wt(d_j)_{i+1}$ correct-instance, is the updated weight of a correctly classified data instance to be trained by the next weak classifier.

$wt(d_j)_{i+1}$ incorrect-instance, is the updated weight of an incorrectly classified data instance to be trained by the next weak classifier.

(Continued)

Data instances that are wrongly classified will have their weights increased.

Data instances that are classified correctly will have their weights decreased.

Step 3: Compute the final predicted value for each data instance. It is the linear combination of weighted average predictions of all the classifiers multiplied by weight α_i computed for each classifier.

$$H_f(d_i) = \text{sign} \left(\sum_{i=1}^M \alpha_i \times H_i(d_i) \right)$$

Example 12.2: Consider a training dataset of six data instances as shown in Table 12.5.

Table 12.5: Training Dataset

CGPA	Interactiveness	Practical Knowledge	Communication Skills	Job Offer
≥ 9	Yes	Good	Good	Yes
< 9	No	Good	Moderate	Yes
≥ 9	No	Average	Moderate	No
< 9	No	Average	Good	No
≥ 9	Yes	Good	Moderate	Yes
≥ 9	Yes	Good	Moderate	Yes

Use 4 Decision Stumps for each of the 4 attributes.

The target attribute 'Job Offer' is a categorical attribute which predicts each data instance as 'Yes' or 'No'.

Number of data instances, $N = 6$.

Apply AdaBoost algorithm here.

Solution:

Step 1: Initial weight assigned to each item = $1/6$.

Step 2: Iterate for each Weak classifier.

I. Decision Stump for CGPA

Step 2 (a): Train the Decision Stump H_{CGPA} with a random bootstrap sample from the training dataset T . Since there are only 6 data instances, use the full training dataset.

The first Decision stump classifies the instances based on the CGPA attribute as shown in Table 12.6. If $CGPA \geq 9$, the data instance is predicted to have 'Job Offer' as 'Yes' else 'No'.

Table 12.6: Decision Stump Prediction using H_{CGPA}

CGPA	Predicted Job Offer	Actual Job Offer
≥ 9	Yes	Yes
< 9	No	Yes
≥ 9	Yes	No
< 9	No	No
≥ 9	Yes	Yes
≥ 9	Yes	Yes

Step 2 (b): Compute the weighted error ϵ_{CGPA} of H_{CGPA} on current training dataset T :

$$\epsilon_i = \sum_{j=1}^N H_i(d_j) wt(d_j) \quad \begin{aligned} H_i(d_j) &= 0 \text{ if prediction is correct with } H_i \\ H_i(d_j) &= 1 \text{ if prediction is wrong with } H_i \end{aligned}$$

$$\epsilon_{CGPA} = 2 \times 1/6 = 0.333$$

Step 2 (c): Compute the weight of each weak classifier:

$$\alpha_{CGPA} = \frac{1}{2} \ln \left(\frac{1 - \epsilon_{CGPA}}{\epsilon_{CGPA}} \right)$$

$$\begin{aligned} \alpha_{CGPA} &= \frac{1}{2} \ln \left(\frac{1 - 0.333}{0.333} \right) \\ &= 0.347 \end{aligned}$$

Step 2 (d): Calculate the normalizing factor Z_{CGPA} .

$$\begin{aligned} Z_{CGPA} &= wt(\text{correctly classified instance}) \times \text{Number of correct classifications} \times e^{-\alpha_{CGPA}} \\ &\quad + wt(\text{incorrectly classified instance}) \times \text{Number of incorrect classifications} \times e^{+\alpha_{CGPA}} \end{aligned}$$

$$\begin{aligned} Z_{CGPA} &= 1/6 \times 4 \times e^{-0.347} + 1/6 \times 2 \times e^{0.347} \\ &= 0.1178 \times 4 + 0.2358 \times 2 \\ &= 0.9428 \end{aligned}$$

Step 2 (e): Update the weight of all data instances:

$$\begin{aligned} wt(d_j)_{i+1} \text{ correct-instance} &= \frac{wt(d_j)_{CGPA} \text{ correct-instance} \times e^{-\alpha_{CGPA}}}{Z_{CGPA}} \\ wt(d_j)_{i+1} \text{ incorrect-instance} &= \frac{wt(d_j)_{CGPA} \text{ incorrect-instance} \times e^{+\alpha_{CGPA}}}{Z_{CGPA}} \\ wt(d_j)_{i+1} \text{ correct-instance} &= \frac{1/6 \times e^{-0.347}}{0.9428} \\ &= \frac{0.1178}{0.9482} = 0.1249 \\ wt(d_j)_{i+1} \text{ incorrect-instance} &= \frac{1/6 \times e^{0.347}}{0.9428} \\ &= \frac{0.2358}{0.9482} = 0.2501 \end{aligned}$$

Table 12.7 shows the modified weights for the data instances after learning with H_{CGPA} .

Table 12.7: Modified Weights with H_{CGPA}

CGPA	Predicted Job Offer	Actual Job Offer	Weights
≥ 9	Yes	Yes	0.1249
< 9	No	Yes	0.2501
≥ 9	Yes	No	0.2501
< 9	No	No	0.1249
≥ 9	Yes	Yes	0.1249
≥ 9	Yes	Yes	0.1249

II. Decision Stump for Interactiveness

Step 2 (a): Train the Decision Stump $H_{Interactiveness}$ with the sample obtained from the previous weak classifier H_{CGPA} .

The second Decision stump classifies the instances based on the Interactiveness attribute as shown in Table 12.8. If Interactiveness = 'Yes', the data instance is predicted to have 'Job Offer' as 'Yes', else if Interactiveness = 'No' the data instance is predicted to have 'Job Offer' as 'No'.

Table 12.8: Decision Stump Prediction using $H_{Interactiveness}$

Interactiveness	Predicted Job Offer	Actual Job Offer	Weights
Yes	Yes	Yes	0.1249
No	No	Yes	0.2501
No	No	No	0.2501
No	No	No	0.1249
Yes	Yes	Yes	0.1249
Yes	Yes	Yes	0.1249

Step 2 (b): Compute the weighted error $\epsilon_{Interactiveness}$ of $H_{Interactiveness}$.

Only one instance is misclassified by this decision stump:

$$\epsilon_{Interactiveness} = 1 \times 0.2501 = 0.2501$$

Step 2 (c): Compute the weight of each weak classifier:

$$\alpha_{Interactiveness} = \frac{1}{2} \ln \left(\frac{1 - \epsilon_{Interactiveness}}{\epsilon_{Interactiveness}} \right)$$

$$\begin{aligned} \alpha_{Interactiveness} &= \frac{1}{2} \ln \left(\frac{1 - 0.2501}{0.2501} \right) \\ &= 0.5490 \end{aligned}$$

Step 2 (d): Calculate the normalizing factor $Z_{Interactiveness}$.

$$\begin{aligned} Z_{Interactiveness} &= wt(\text{correctly classified instance}) \times \text{Number of correct classifications} \times e^{-\alpha_{Interactiveness}} \\ &\quad + wt \left(\begin{array}{c} \text{incorrectly} \\ \text{classified instance} \end{array} \right) \times \text{Number of incorrect classifications} \times e^{+\alpha_{Interactiveness}} \end{aligned}$$

$$\begin{aligned}
 Z_{\text{Interactiveness}} &= 0.1249 \times 4 \times e^{-0.5490} + 0.2501 \times 1 \times e^{0.5490} + 0.2501 \times 1 \times e^{-0.5490} \\
 &= 0.2885 + 0.4331 + 0.1444 \\
 &= 0.866
 \end{aligned}$$

Step 2 (e): Update the weight of all data instances:

$$\begin{aligned}
 wt(d_j)_{i+1} \text{ correct-instance} &= \frac{wt(d_j)_{\text{Interactiveness}} \text{ correct-instance} \times e^{-\alpha_{\text{Interactiveness}}}}{Z_{\text{Interactiveness}}} \\
 wt(d_j)_{i+1} \text{ incorrect-instance} &= \frac{wt(d_j)_{\text{Interactiveness}} \text{ incorrect-instance} \times e^{\alpha_{\text{Interactiveness}}}}{Z_{\text{Interactiveness}}} \\
 wt(d_j)_{i+1} \text{ correct-instance} &= \frac{0.1249 \times e^{-0.5490}}{0.866} \\
 &= \frac{0.072}{0.866} = 0.0832 \\
 wt(d_j)_{i+1} \text{ incorrect-instance} &= \frac{0.2501 \times e^{0.5490}}{0.866} \\
 &= \frac{0.4331}{0.866} = 0.5001 \\
 wt(d_j)_{i+1} \text{ correct-instance} &= \frac{0.2501 \times e^{-0.5490}}{0.866} \\
 &= \frac{0.1444}{0.866} = 0.1667
 \end{aligned}$$

Table 12.9 shows the modified weights for the data instances after learning with $H_{\text{Interactiveness}}$.

Table 12.9: Modified Weights with $H_{\text{Interactiveness}}$

Interactiveness	Predicted Job Offer	Actual Job Offer	Updated Weights
Yes	Yes	Yes	0.0832
No	No	Yes	0.5001
No	No	No	0.1667
No	No	No	0.0832
Yes	Yes	Yes	0.0832
Yes	Yes	Yes	0.0832

III. Decision Stump (Practical Knowledge)

Step 2 (a): Train the Decision Stump $H_{\text{Practical Knowledge}}$ with the sample obtained from the previous weak classifier $H_{\text{Interactiveness}}$.

The third Decision stump classifies the instances based on the Practical Knowledge attribute as shown in Table 12.10. If Practical Knowledge = 'Good' the data instance is predicted to have 'Job Offer' as 'Yes', else if Practical Knowledge = 'Average' the data instance is predicted to have 'Job Offer' as 'No'.

Table 12.10: Decision Stump Prediction using $H_{\text{Practical Knowledge}}$

Practical Knowledge	Actual Job Offer	Predicted Job Offer	Weights
Good	Yes	Yes	0.0832
Good	Yes	Yes	0.5001
Average	No	No	0.1667
Average	No	No	0.0832
Good	Yes	Yes	0.0832
Good	Yes	Yes	0.0832

No instances are misclassified by this Decision Stump. So, don't change the weights of the data instances.

IV. Decision Stump (Communication Skills)

Step 2 (a): Train the Decision Stump $H_{\text{Communication Skills}}$ with the sample obtained from the previous weak classifier $H_{\text{Interactiveness}}$.

The fourth Decision stump classifies the instances based on the Communication Skills attribute as shown in Table 12.11. If Communication Skills = 'Good' the data instance is predicted to have 'Job Offer' as 'Yes', else if Communication Skills = 'Moderate' the data instance is predicted to have 'Job Offer' as 'No'.

Table 12.11: Decision Stump Prediction using $H_{\text{Communication Skills}}$

Communication Skills	Predicted Job Offer	Actual Job Offer	Weights
Good	Yes	Yes	0.0832
Moderate	No	Yes	0.5001
Moderate	No	No	0.1667
Good	Yes	No	0.0832
Moderate	No	Yes	0.0832
Moderate	No	Yes	0.0832

Step 2 (b): Compute the weighted error $\epsilon_{\text{Communication Skills}}$ of $H_{\text{Communication Skills}}$.

Four instances are misclassified by this decision stump:

$$\epsilon_{\text{Communication Skills}} = 1 \times 0.5001 + 3 \times 0.0832 = 0.7497$$

Step 2 (c): Compute the weight of each weak classifier:

$$\alpha_{\text{Communication Skills}} = \frac{1}{2} \ln \left(\frac{1 - \epsilon_{\text{Communication Skills}}}{\epsilon_{\text{Communication Skills}}} \right)$$

$$\begin{aligned} \alpha_{\text{Communication Skills}} &= \frac{1}{2} \ln \left(\frac{1 - 0.7497}{0.7497} \right) \\ &= -0.5485 \end{aligned}$$

Step 2 (d): Calculate the normalizing factor $Z_{\text{Communication Skills}}$.

$$\begin{aligned} Z_{\text{Communication Skills}} &= wt(\text{correctly classified instance}) \times \text{Number of correct classifications} \times e^{-\alpha_{\text{Communication Skills}}} \\ &\quad + wt(\text{incorrectly classified instance}) \times \text{Number of incorrect classifications} \times e^{+\alpha_{\text{Communication Skills}}} \end{aligned}$$

$$\begin{aligned}
Z_{\text{Communication Skills}} &= 0.0832 \times 1 \times e^{-(-0.5485)} + 0.1667 \times 1 \times e^{-(-0.5485)} + 0.0832 \times 3 \times e^{+(-0.5485)} + \\
&\quad 0.5001 \times 1 \times e^{+(-0.5485)} \\
&= 0.1440 + 0.2885 + 0.1442 + 0.2889 \\
&= 0.866
\end{aligned}$$

Step 2 (e): Update the weight of all data instances:

$$wt(d_j)_{i+1} \text{ correct-instance} = \frac{wt(d_j)_{\text{Communication Skills}} \text{correct-instance} \times e^{-\alpha_{\text{Communication Skills}}}}{Z_{\text{Communication Skills}}}$$

$$wt(d_j)_{i+1} \text{ incorrect-instance} = \frac{wt(d_j)_{\text{Communication Skills}} \text{incorrect-instance} \times e^{+\alpha_{\text{Communication Skills}}}}{Z_{\text{Communication Skills}}}$$

$$\begin{aligned}
wt(d_j)_{i+1} \text{ correct-instance} &= \frac{0.0832 \times e^{-(-0.5485)}}{0.866} \\
&= 0.1663
\end{aligned}$$

$$\begin{aligned}
wt(d_j)_{i+1} \text{ correct-instance} &= \frac{0.1667 \times e^{-(-0.5485)}}{0.866} \\
&= 0.3331
\end{aligned}$$

$$\begin{aligned}
wt(d_j)_{i+1} \text{ incorrect-instance} &= \frac{0.5001 \times e^{+(-0.5485)}}{0.866} \\
&= 0.3337
\end{aligned}$$

$$\begin{aligned}
wt(d_j)_{i+1} \text{ incorrect-instance} &= \frac{0.0832 \times e^{+(-0.5485)}}{0.866} \\
&= 0.0555
\end{aligned}$$

Table 12.12 shows the modified weights for the data instances after learning with $H_{\text{Communication Skills}}$.

Table 12.12: Modified Weights with $H_{\text{Communication Skills}}$

Communication Skills	Predicted Job Offer	Actual Job Offer	Updated Weights
Good	Yes	Yes	0.1663
Moderate	No	Yes	0.3337
Moderate	No	No	0.3331
Good	Yes	No	0.0555
Moderate	No	Yes	0.0555
Moderate	No	Yes	0.0555

We can observe that the weights of misclassified instances are increased.

Step 3: Compute the final predicted value for each data instance:

$$H_F(d_j) = \text{sign} \left(\sum_{i=1}^M \alpha_i \times H_i(d_j) \right)$$

The sign function classifies instances as positive if greater than 0 and negative if less than 0. A value of 0 is considered neither positive nor negative.

The final prediction for data instance 1 is computed as shown below,

$$\begin{aligned}
 H_F(d_{j=1}) &= \alpha_{\text{CGPA}} \times \text{Yes} + \alpha_{\text{Interactiveness}} \times \text{Yes} + \alpha_{\text{Communication Skills}} \times \text{Yes} \\
 &= 0.347 \times 1 + 0.5490 \times 1 + -0.5485 \times 1 = 0.3475 \\
 H_F(d_{j=1}) &= \text{Yes}
 \end{aligned}$$

This instance is classified with 'Job Offer = Yes' since the weighted average is positive.

Table 12.13 shows the final predicted value for all the data instances in the training dataset. It can be observed that the algorithm does not over-fit with the training dataset.

Table 12.13: Final Prediction

Instances	$\alpha_{\text{CGPA}} = 0.347$	$\alpha_{\text{Interactiveness}} = 0.5490$	$\alpha_{\text{Communication Skills}} = -0.5485$	Weighted Average	Final Prediction
1.	Yes	Yes	Yes	0.3475	Yes
2.	No	No	No	0	No
3.	Yes	No	No	0.347	Yes
4.	No	No	Yes	-0.5485	No
5.	Yes	Yes	No	0.896	Yes
6.	Yes	Yes	No	0.896	Yes

Advantages of AdaBoost Algorithm

1. Fast
2. Simple to implement

Disadvantages of AdaBoost Algorithm

1. From empirical evidence and theoretical evidence, AdaBoost may overfit
2. Sensitive to noise data and outliers

Scan for information on 'Gradient Tree Boosting' and 'XGBoost'



Summary

1. Ensemble learning is a machine learning model which combines the predictions of several classifiers in order to improve the prediction accuracy of using a single classifier.
2. Bagging and Boosting are two popular models of ensemble learning.
3. A simple way of ensemble learning is majority voting or taking an average or weighted average of predictions of multiple machine learning models or base learners.
4. Ensembling machine learning is an approach to minimize bias and variance, and to build accurate models that avoid over-fitting and under-fitting of learning models.
5. Bagging is otherwise called as bootstrap aggregation, which combines both bootstrapping and aggregation.

6. The random forest ensemble model is a variant of Bagging which uses decision tree for learning.
7. Boosting algorithms train multiple weak learners sequentially and the error exhibited by a weak learner is used to give importance to data instances which are further retrained by the next weak learner.
8. AdaBoost, called as adaptive boosting, is a popular boosting algorithm that trains multiple weak classifiers sequentially so that the errors made by poorly performing weak classifiers are reduced when they are trained sequentially.
9. AdaBoost is a greedy algorithm optimizing weights of data instances in the training dataset by adding a weak classifier at each step.

Key Terms

- **Stacking** – A method of ensembling that uses the predictions of multiple models as 'features' to train a new model and predict from the new model.
- **Bagging** – A method of ensembling simultaneously many independent models and averaging the predictions obtained from different models.
- **Boosting** – Employs different classifiers in a sequential fashion where the prediction made by a classifier is enhanced by boosting the weights for the data instances.
- **Parallel Ensemble Model** – Enables the base learners to learn simultaneously.
- **Sequential Ensemble Model** – Enables the base learners to learn sequentially.
- **Voting** – A process of combining multiple machine learning models or using the same model multiple times as base-learners and taking average of their predictions in the case of regression problem and considering the maximum vote of their predictions in the case of classification problem.
- **Bootstrap Resampling** – A process of creating many bootstrap samples S_i ($1 \leq i \leq M$) using a method called Random Sampling with replacement from the training dataset.
- **Cascading Ensemble Model** – It is called as cascading as learners perform multistage learning with different machine learning models and go to next stage of learning only if the preceding learners' prediction is not confident.

Review Questions

1. What is meant by ensembling? What is the need for ensembling multiple learners?
2. What are the advantages and disadvantages of ensemble learning?
3. How can we classify the various ensemble learning techniques?
4. Compare bagging and boosting.
5. What is the principle behind Random forest ensemble model?
6. What is the general idea of boosting techniques?
7. Compare AdaBoost and gradient tree boosting.
8. What features are added to XGBoost to make it perform better than other boosting algorithms?
9. Consider the following student performance dataset shown in Table 12.5. Apply the AdaBoost algorithm to train a classifier for the problem.